LAPORAN PRAKTIKUM

POSTTEST 6

ALGORITMA PEMROGRAMAN DASAR



Disusun oleh:

M Al Kahfi Hafidz M 2609106013

Kelas A1 ‘26

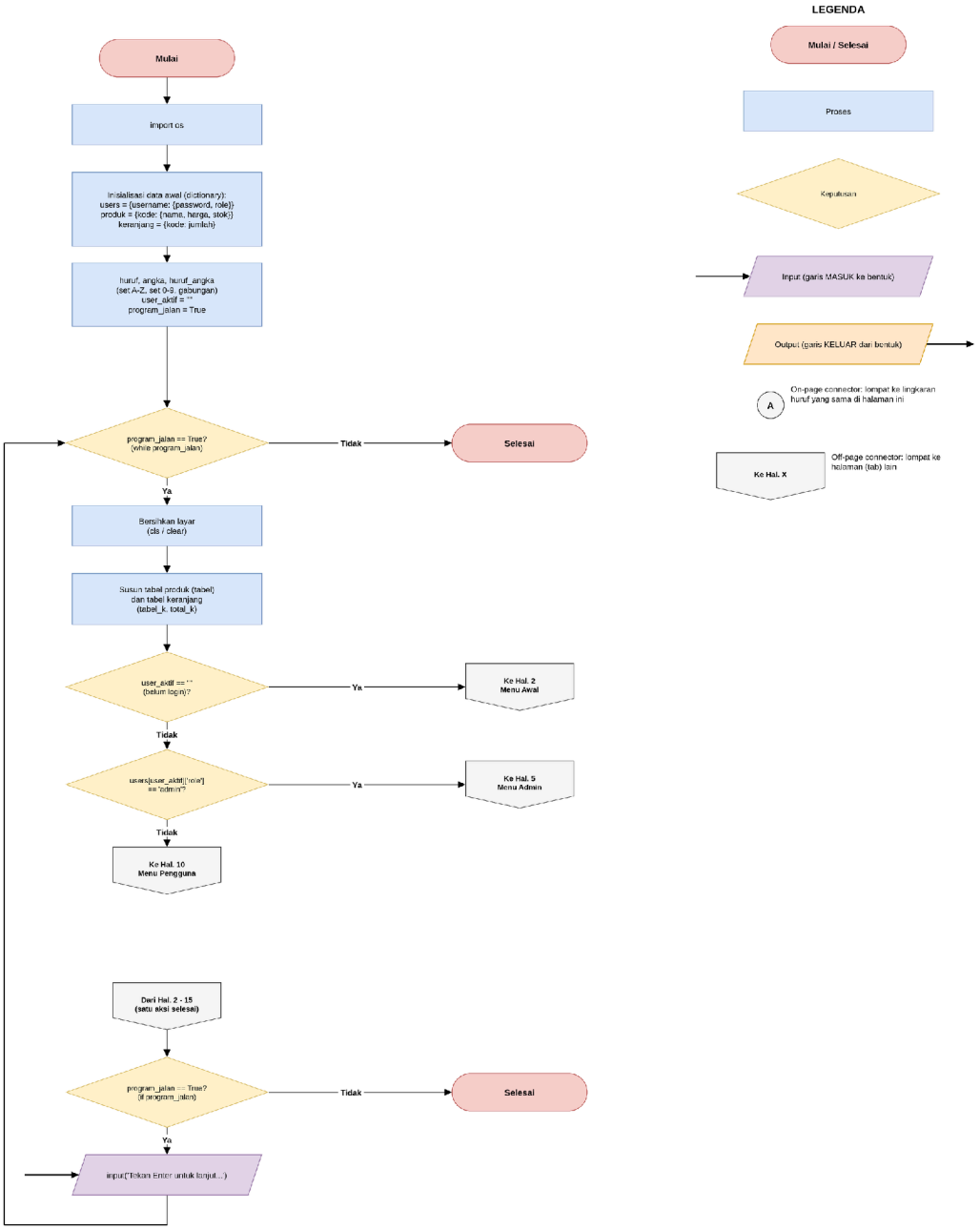
PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

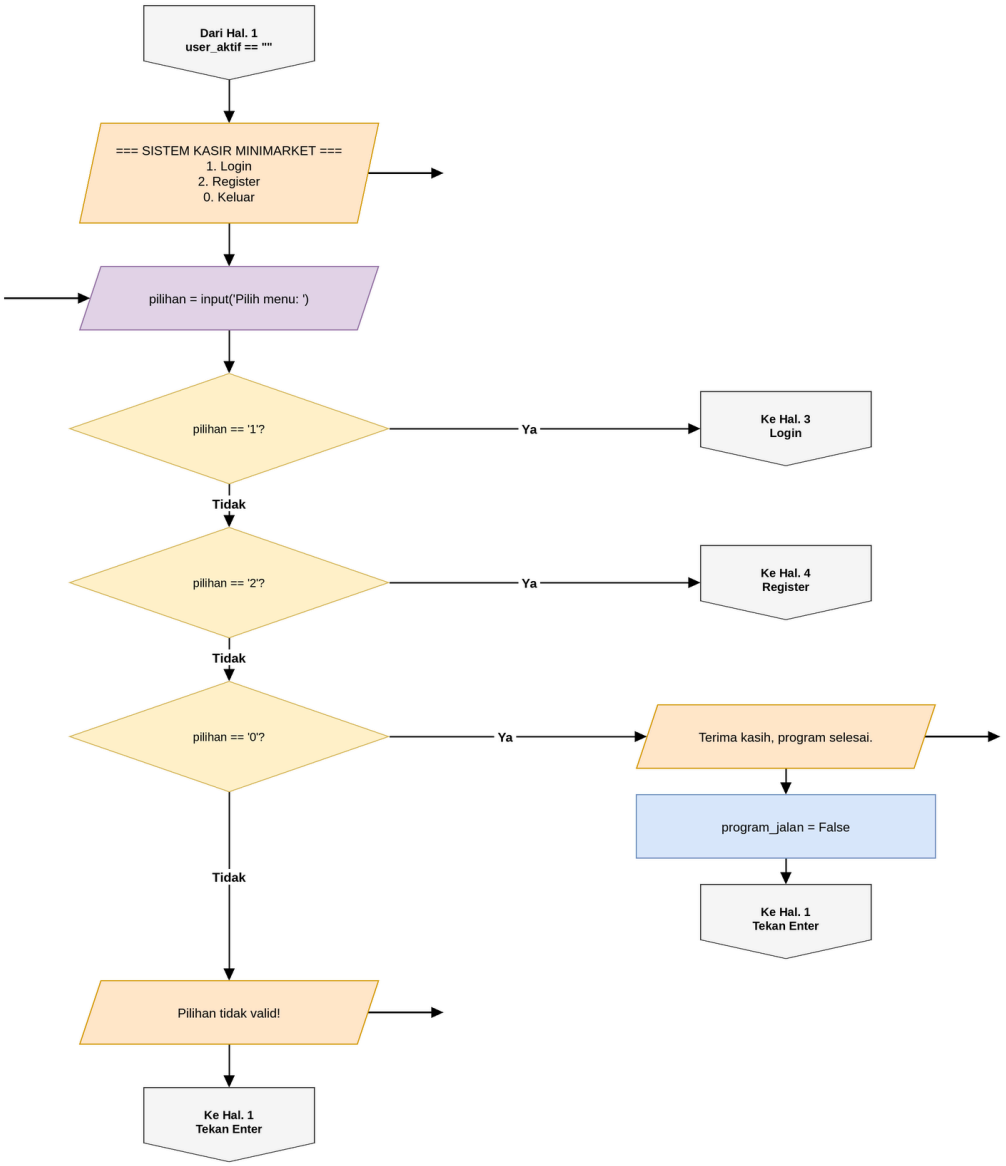
SAMARINDA

2026

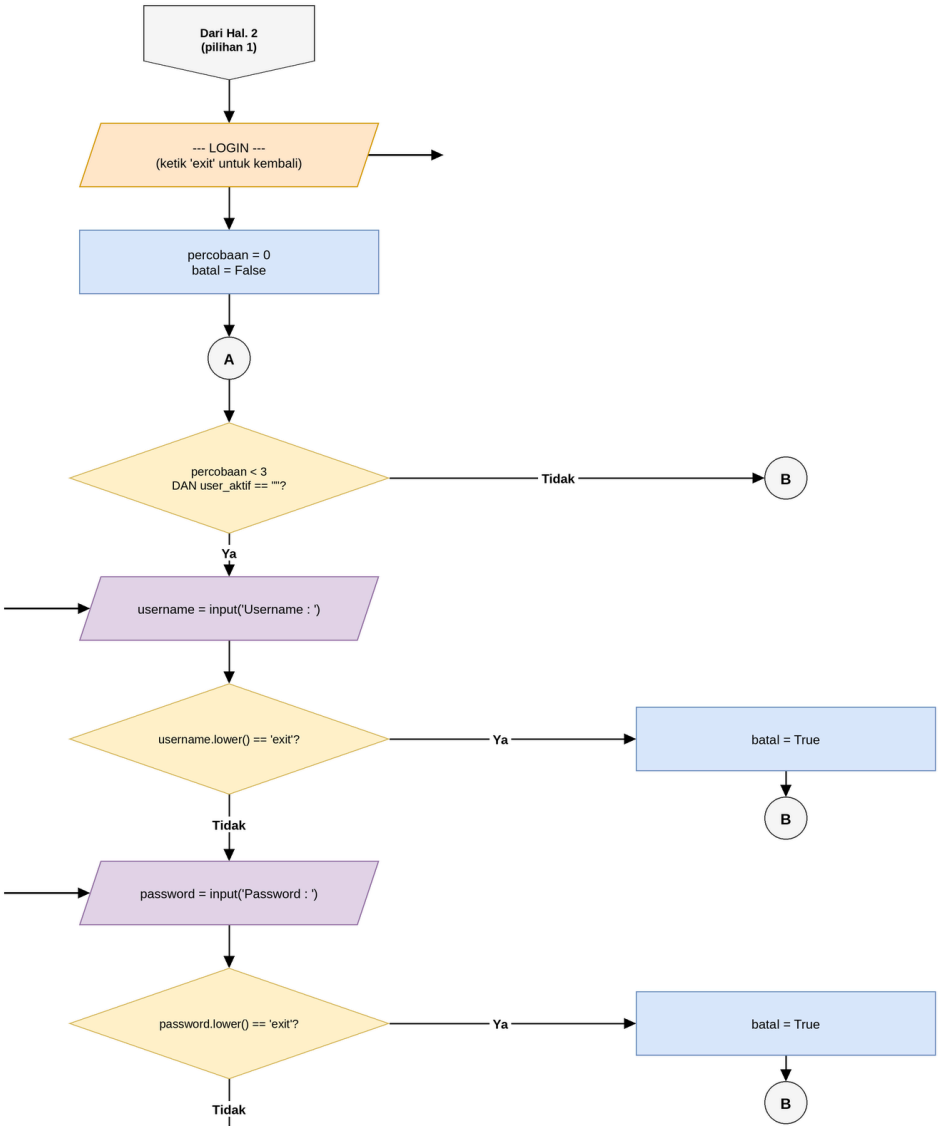
1. Flowchart



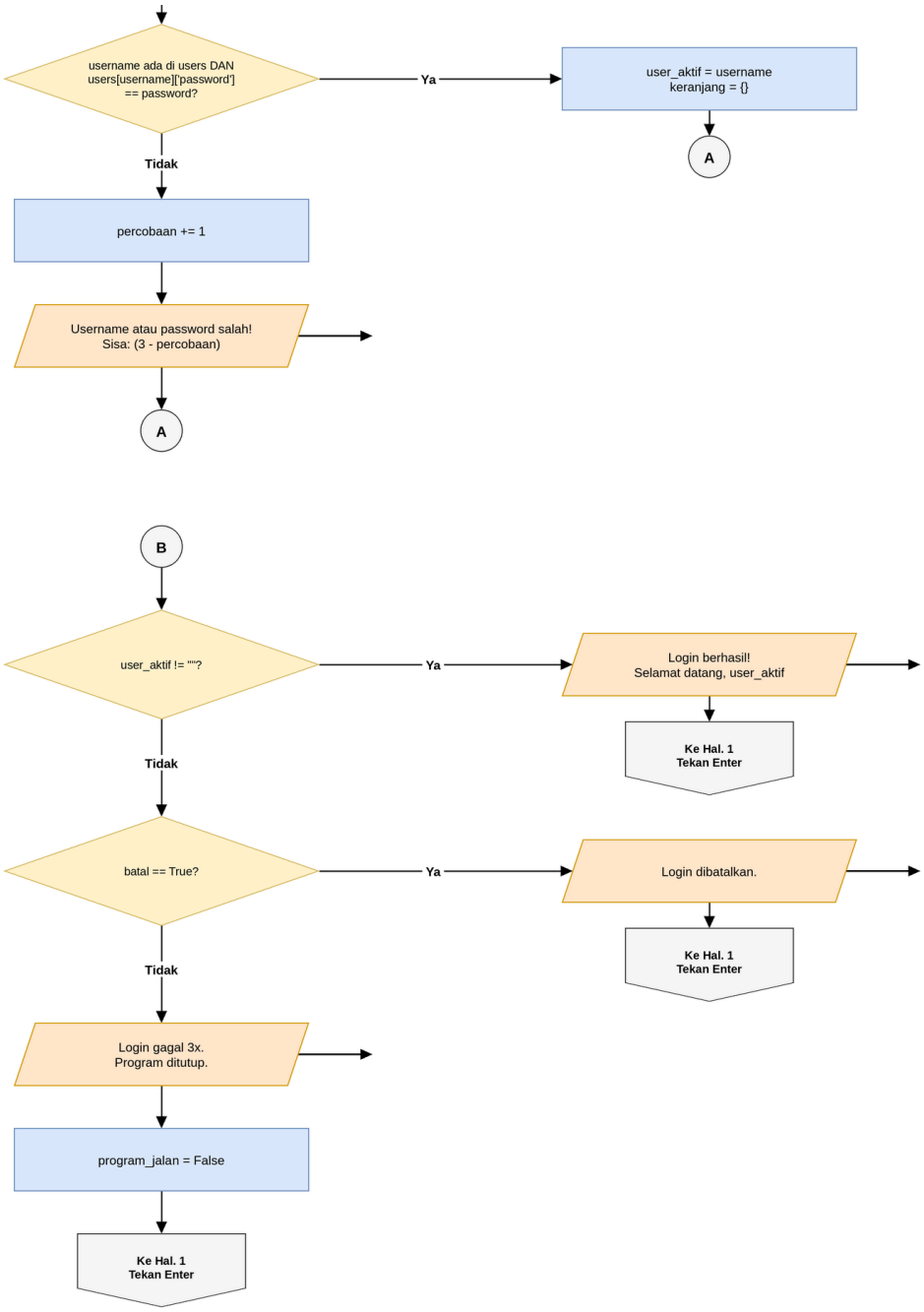
Gambar 1.1 *Flowchart Program Utama (Loop)*



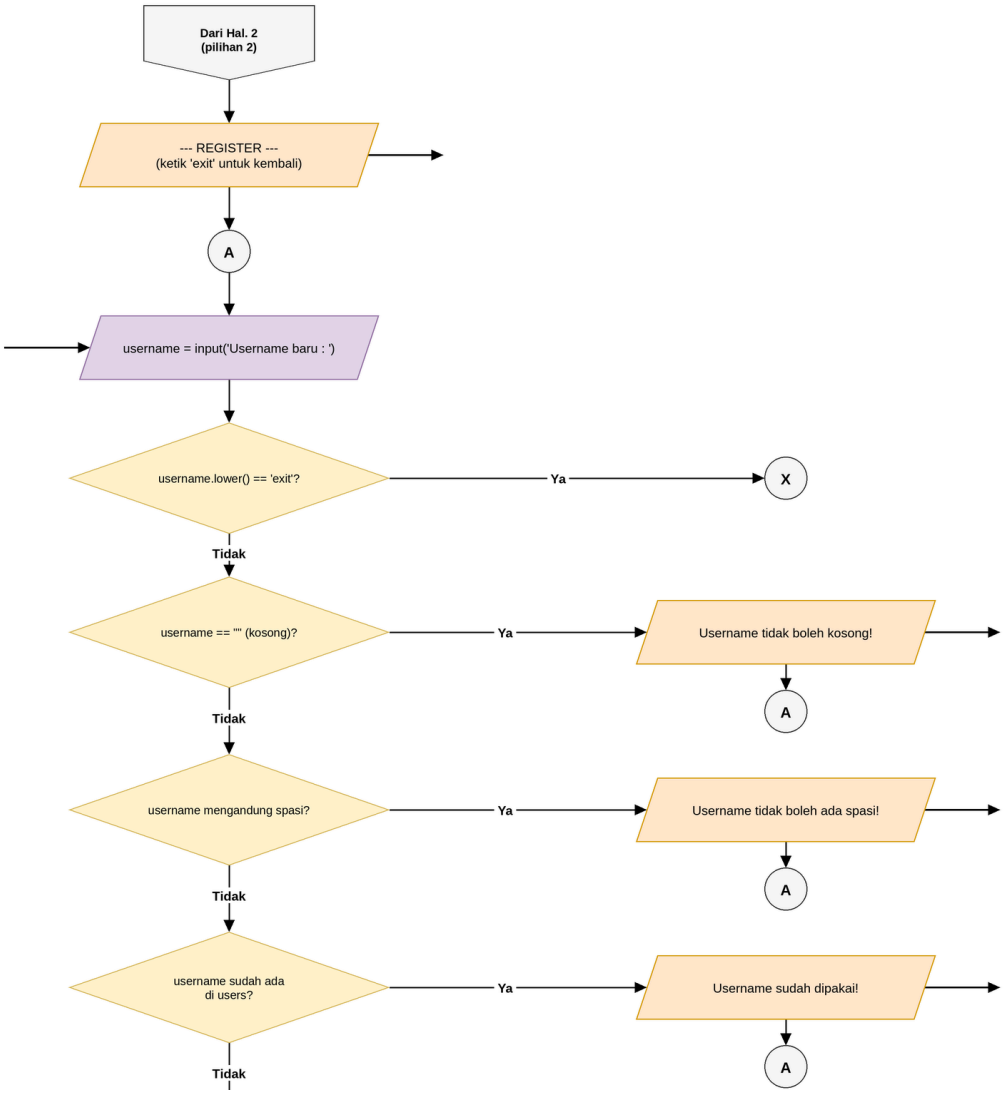
Gambar 1.2 *Flowchart Menu Awal*



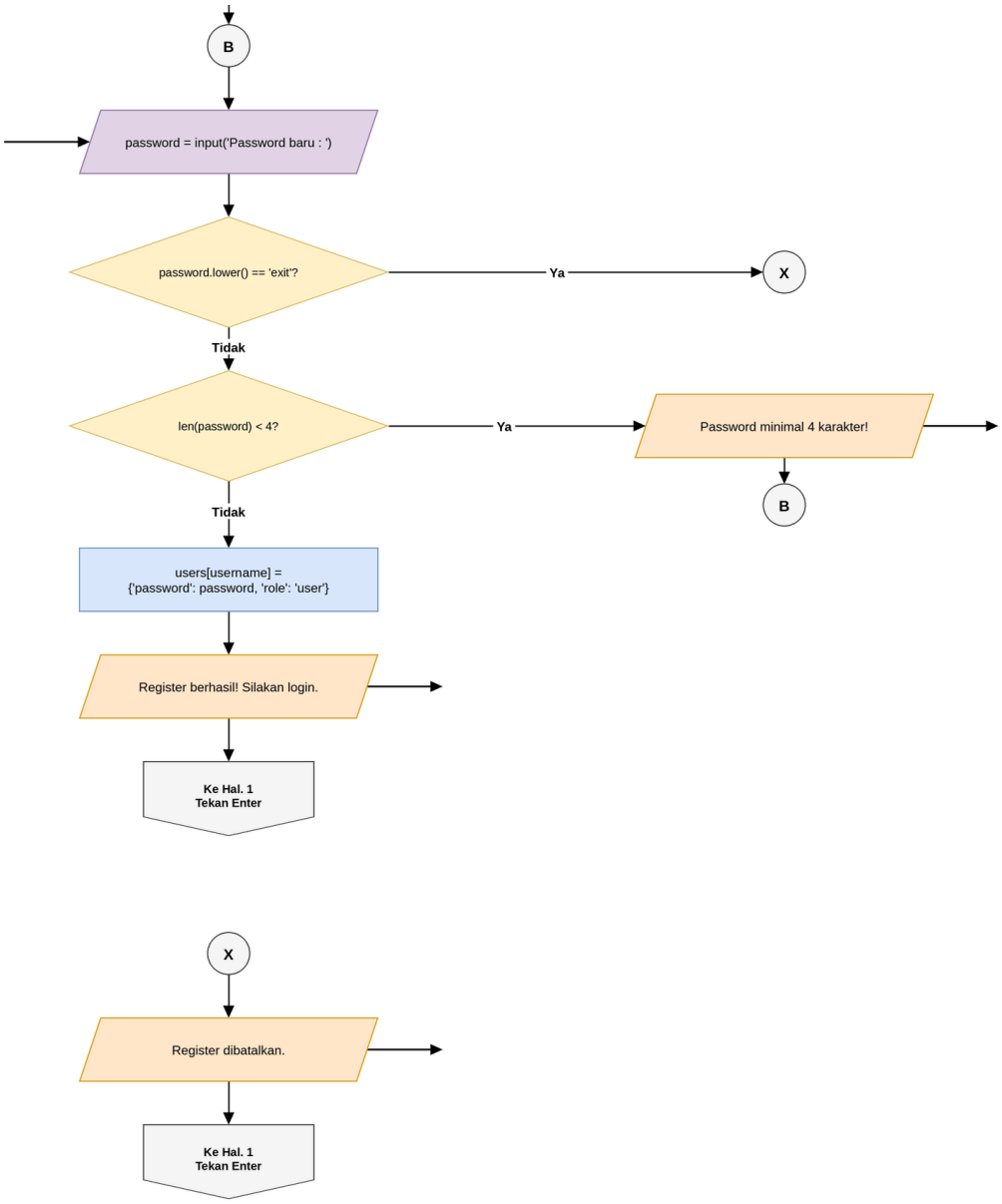
Gambar 1.3 *Flowchart Login (bagian 1 dari 2)*



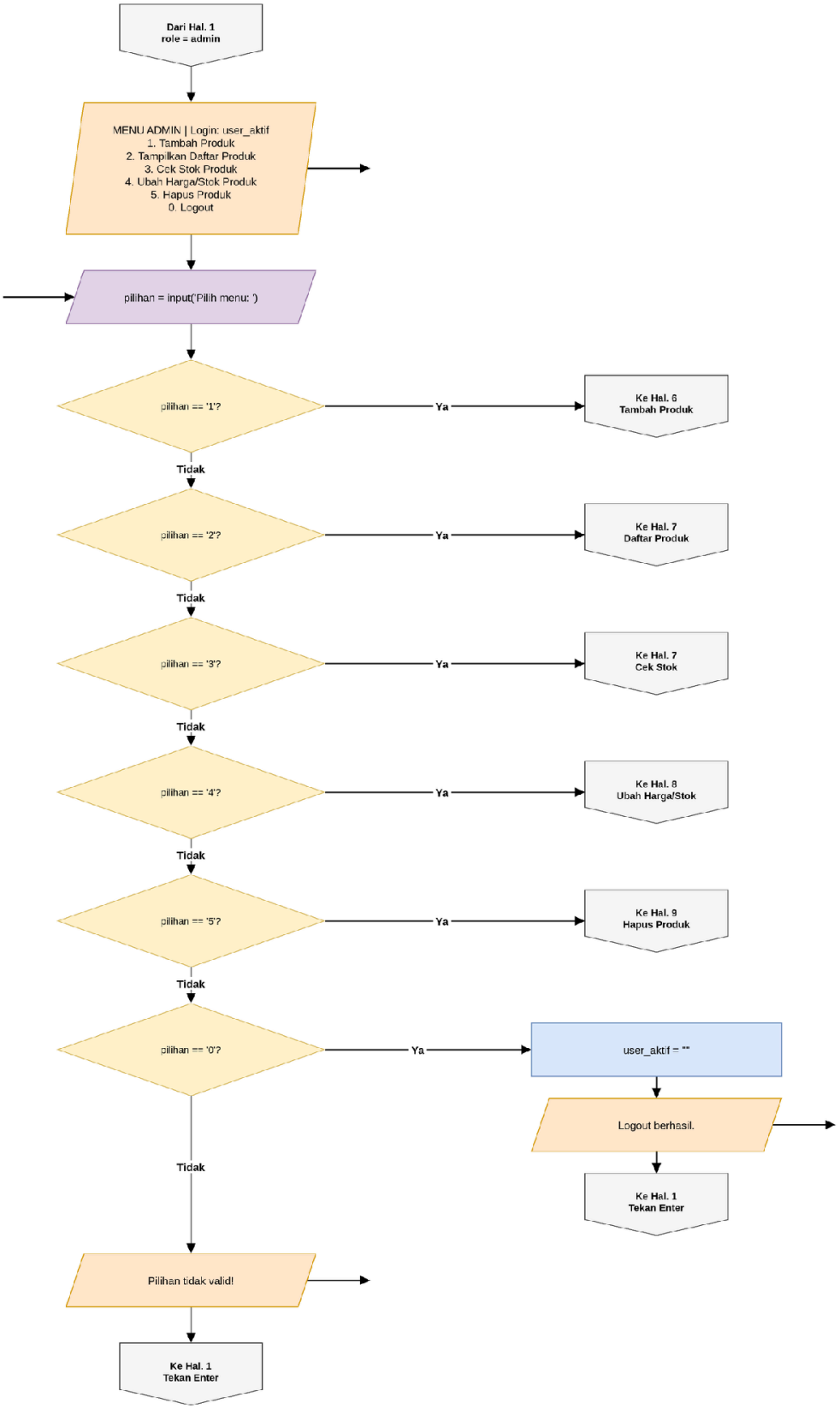
Gambar 1.3 *Flowchart Login (bagian 2 dari 2)*



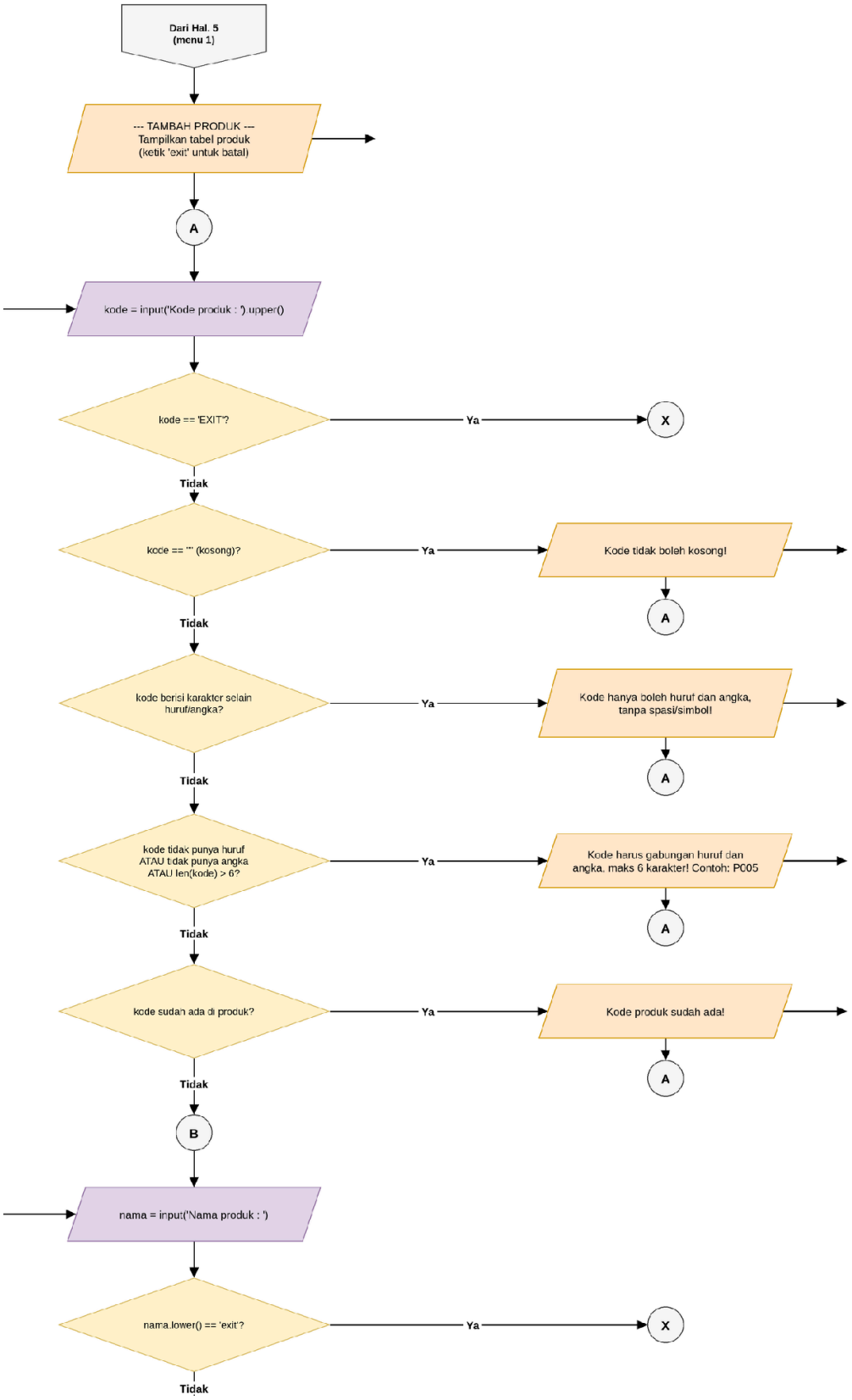
Gambar 1.4 *Flowchart Register (bagian 1 dari 2)*



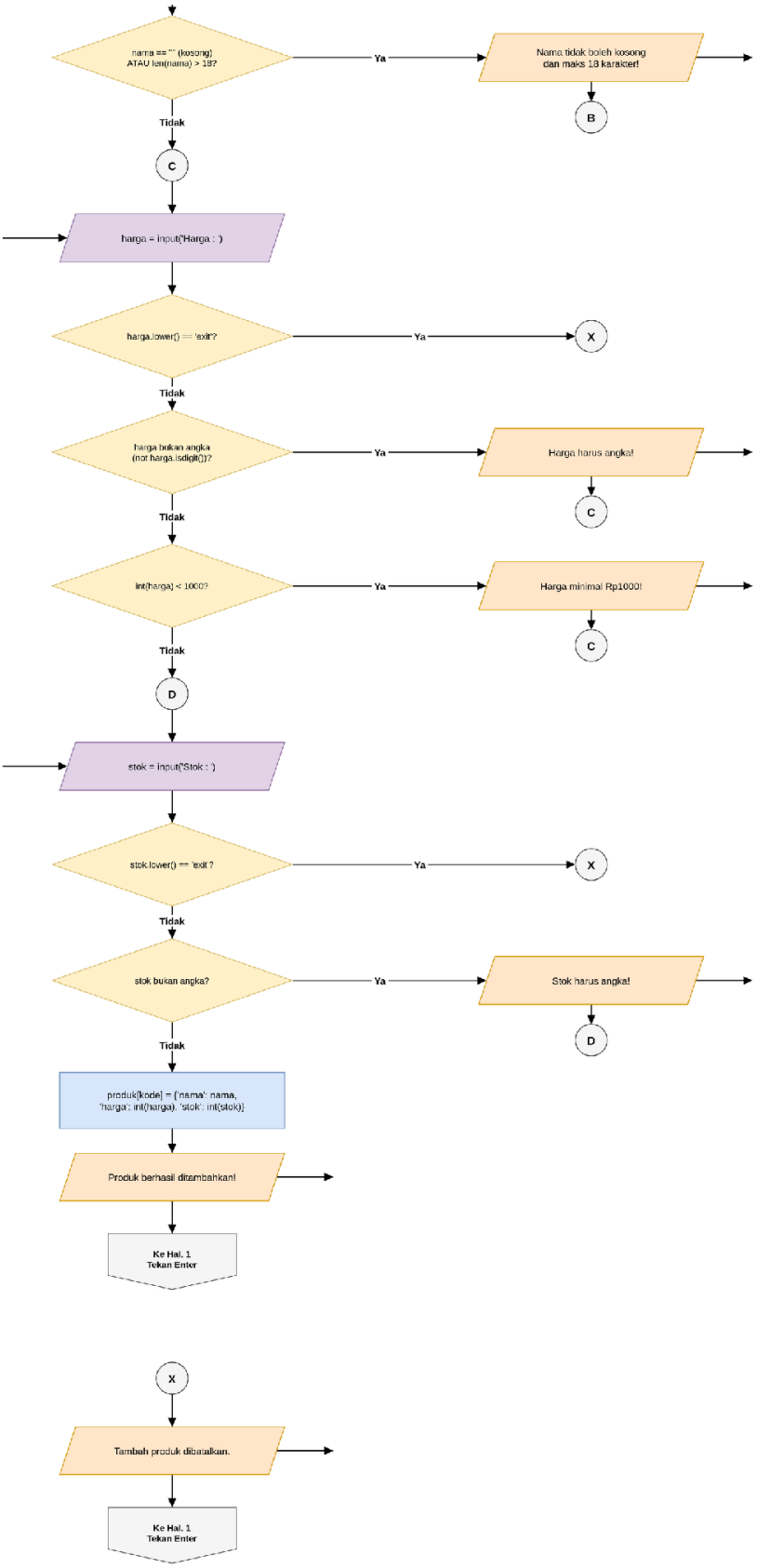
Gambar 1.4 *Flowchart Register (bagian 2 dari 2)*



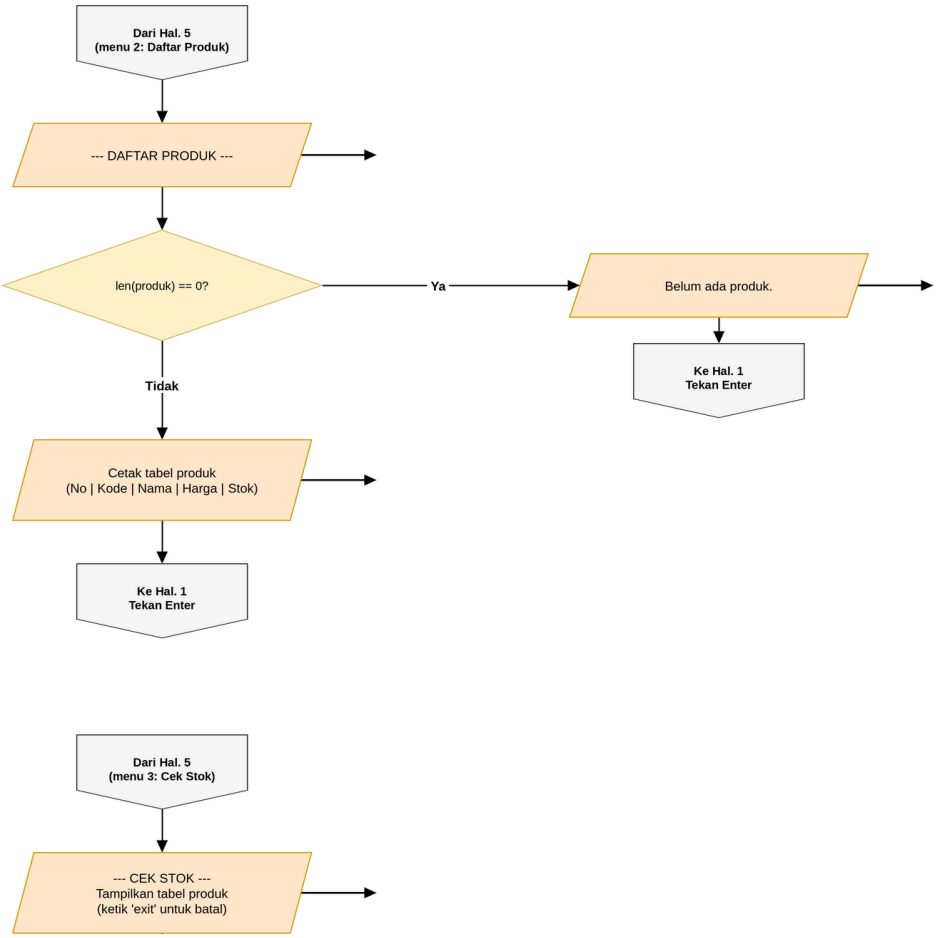
Gambar 1.5 *Flowchart Menu Admin*



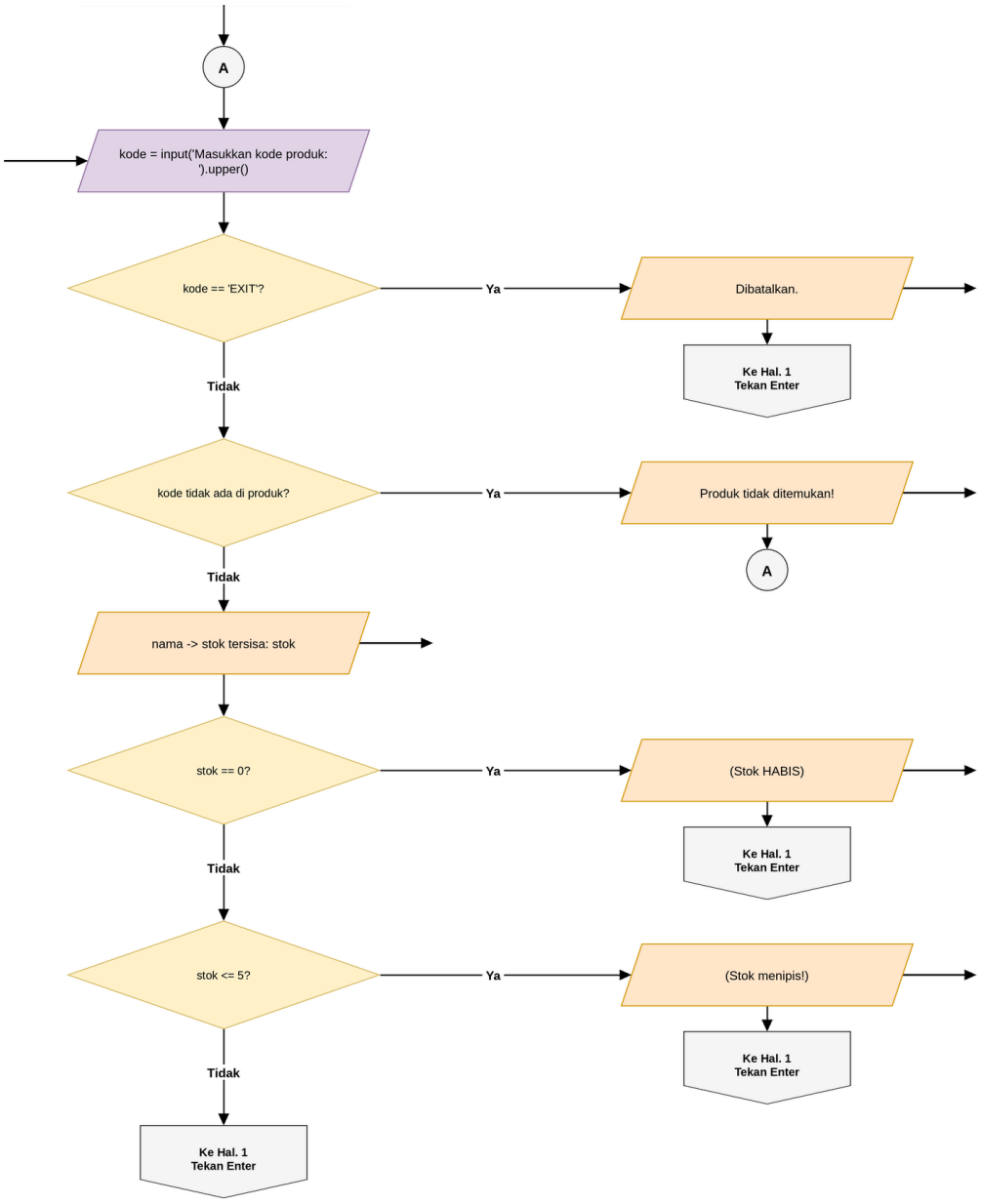
Gambar 1.6 *Flowchart Admin - Tambah Produk (bagian 1 dari 2)*



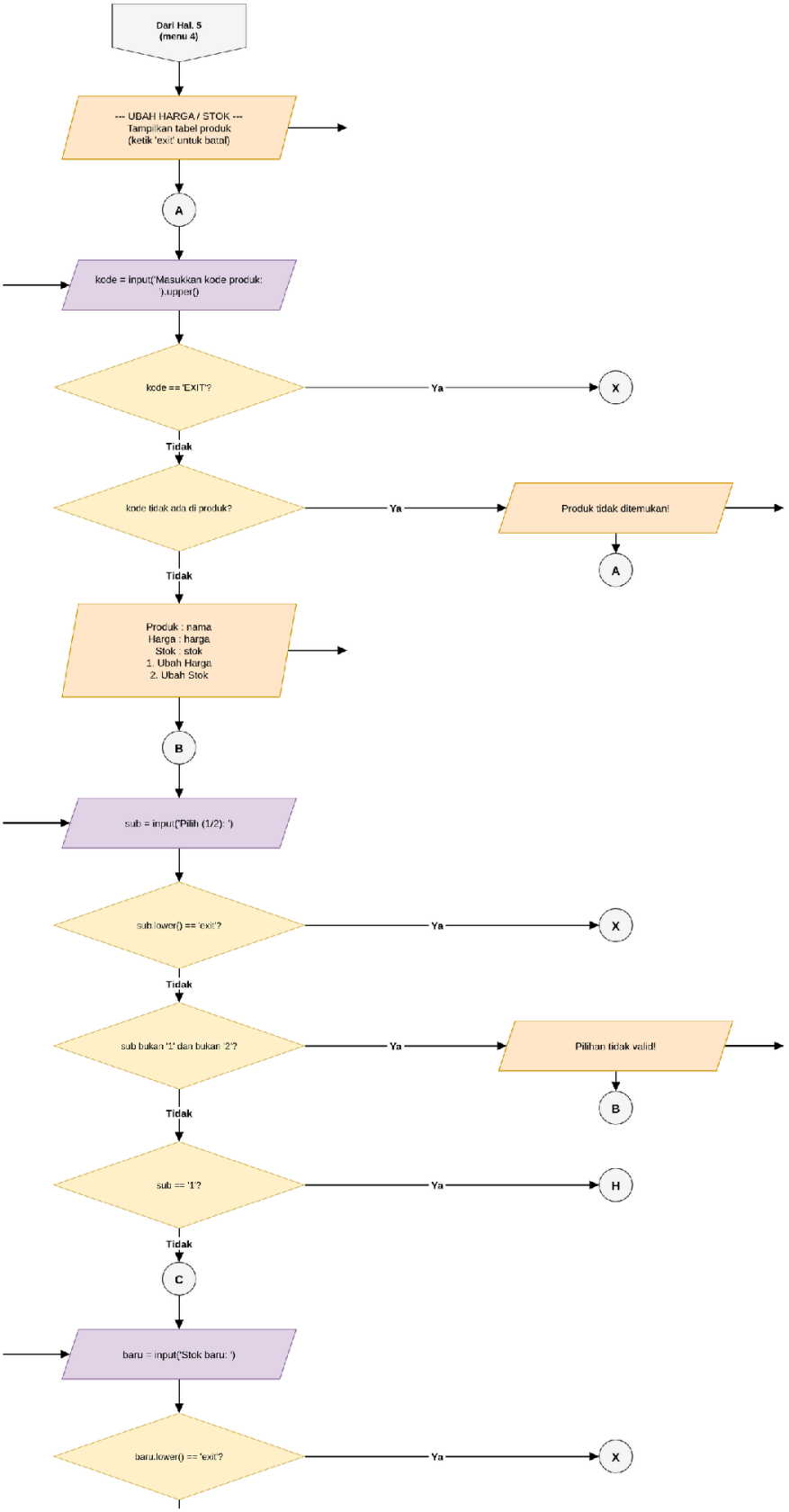
Gambar 1.6 *Flowchart Admin - Tambah Produk (bagian 2 dari 2)*



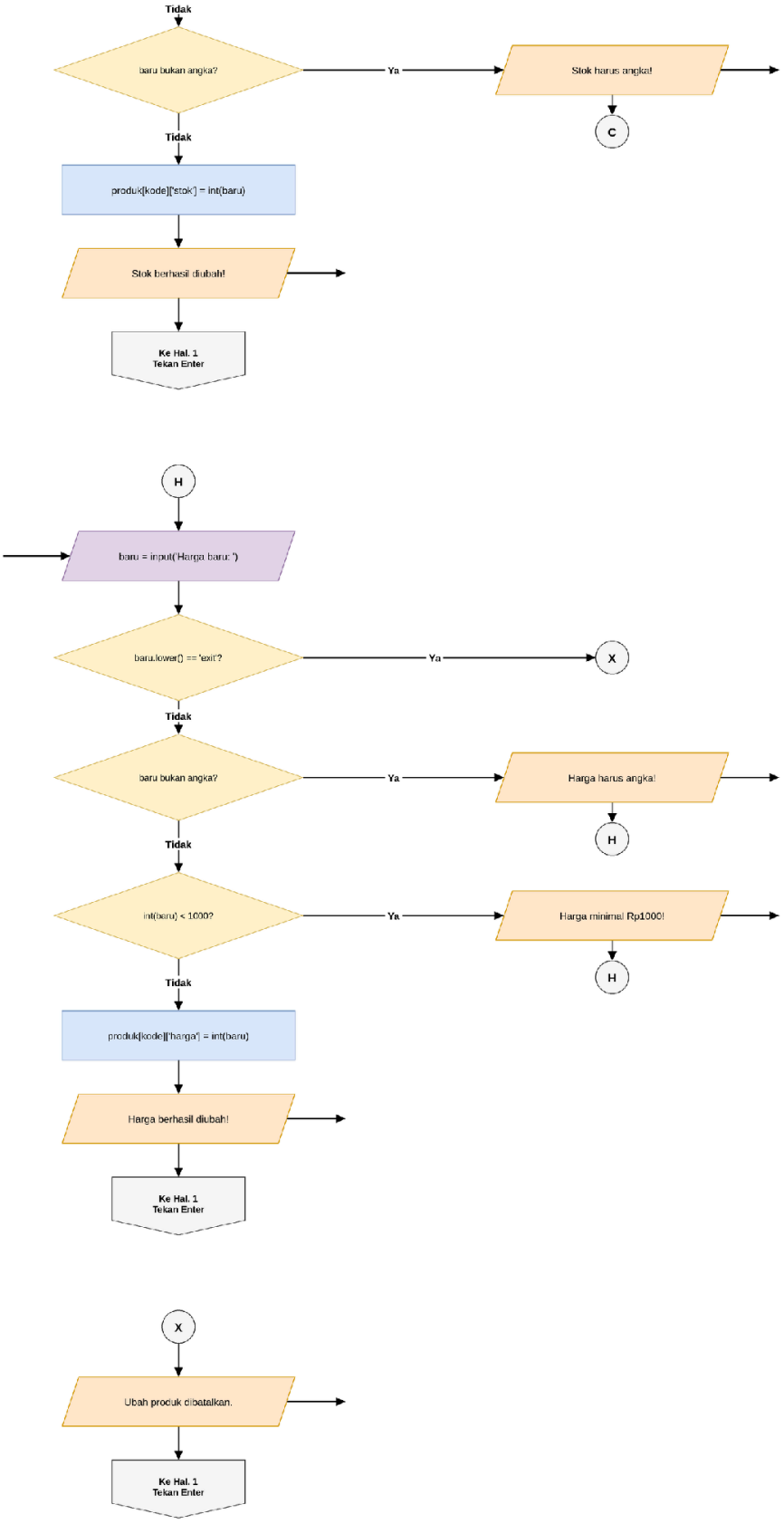
Gambar 1.7 *Flowchart Admin - Daftar & Cek Stok (bagian 1 dari 2)*



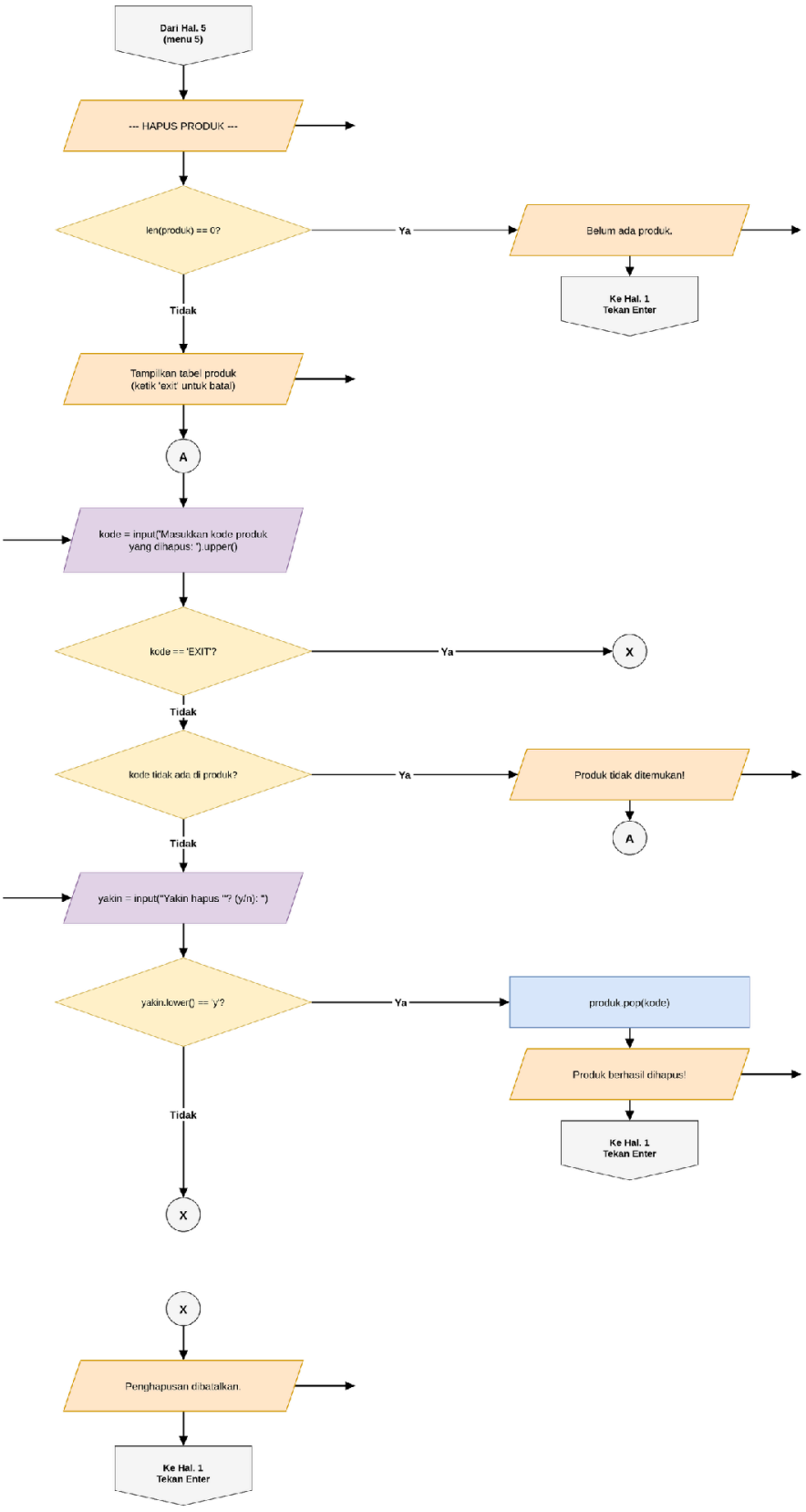
Gambar 1.7 *Flowchart Admin - Daftar & Cek Stok (bagian 2 dari 2)*



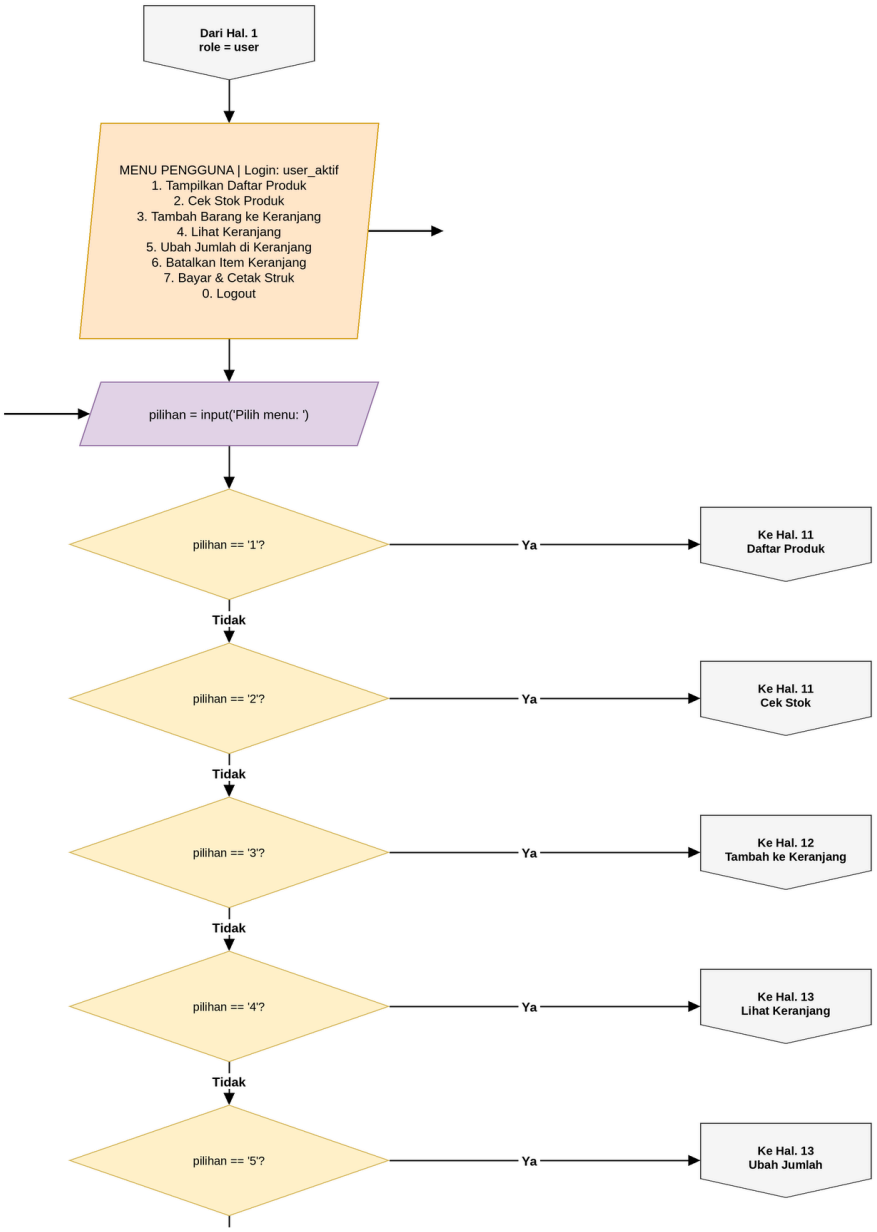
Gambar 1.8 *Flowchart Admin - Ubah Harga-Stok (bagian 1 dari 2)*



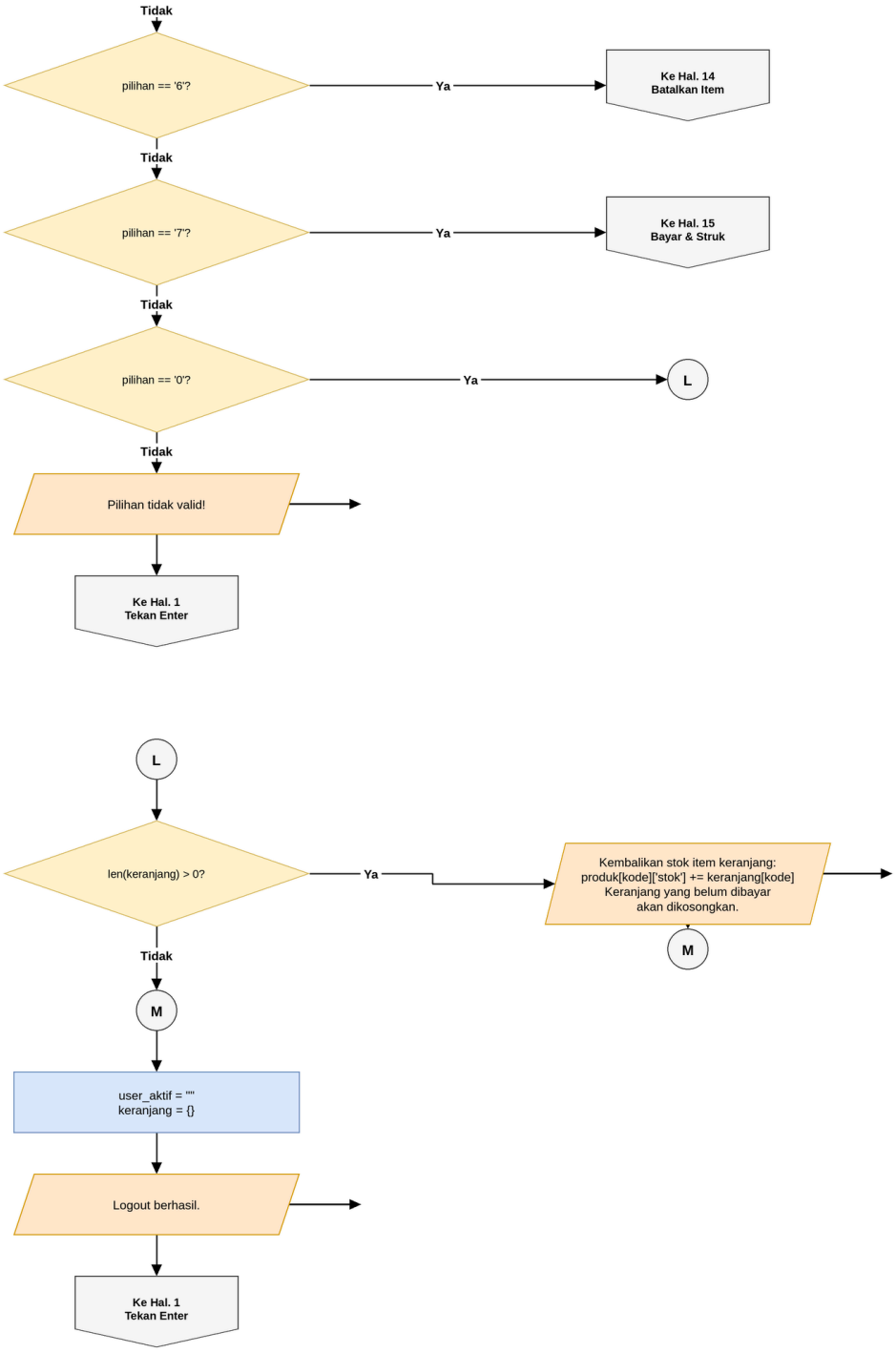
Gambar 1.8 *Flowchart Admin - Ubah Harga-Stok (bagian 2 dari 2)*



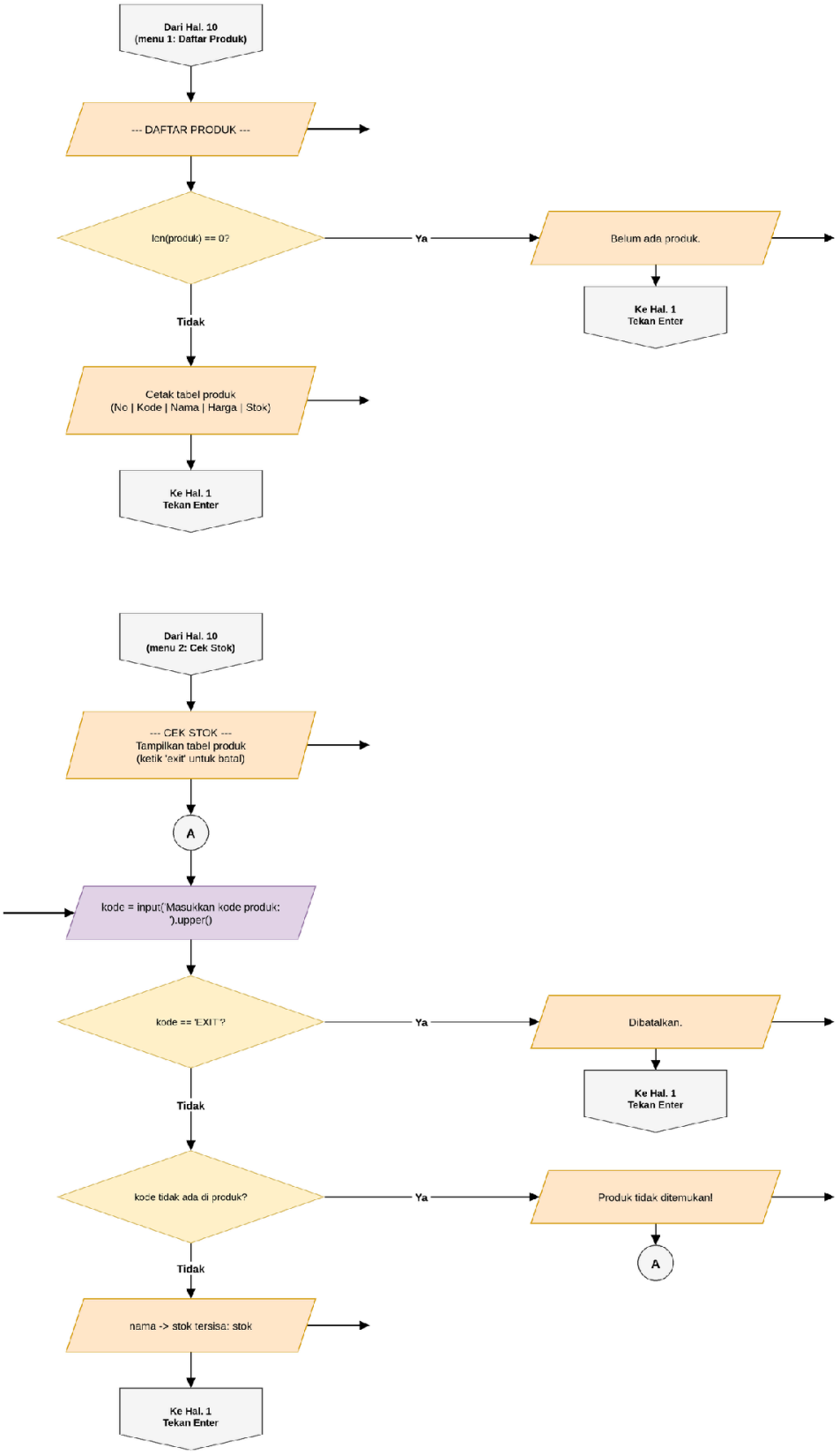
Gambar 1.9 *Flowchart Admin - Hapus Produk*



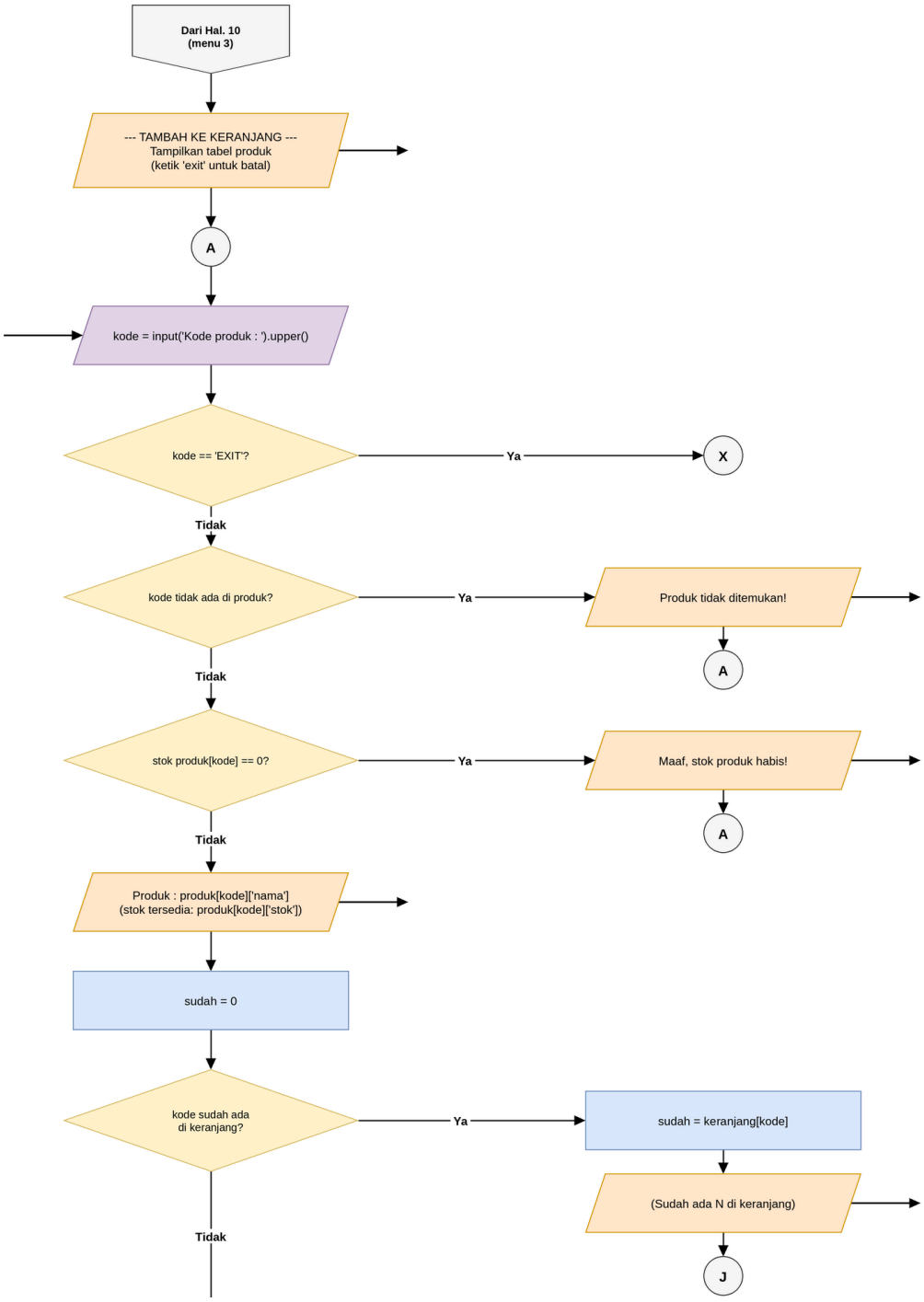
Gambar 1.10 *Flowchart Menu Pengguna (bagian 1 dari 2)*



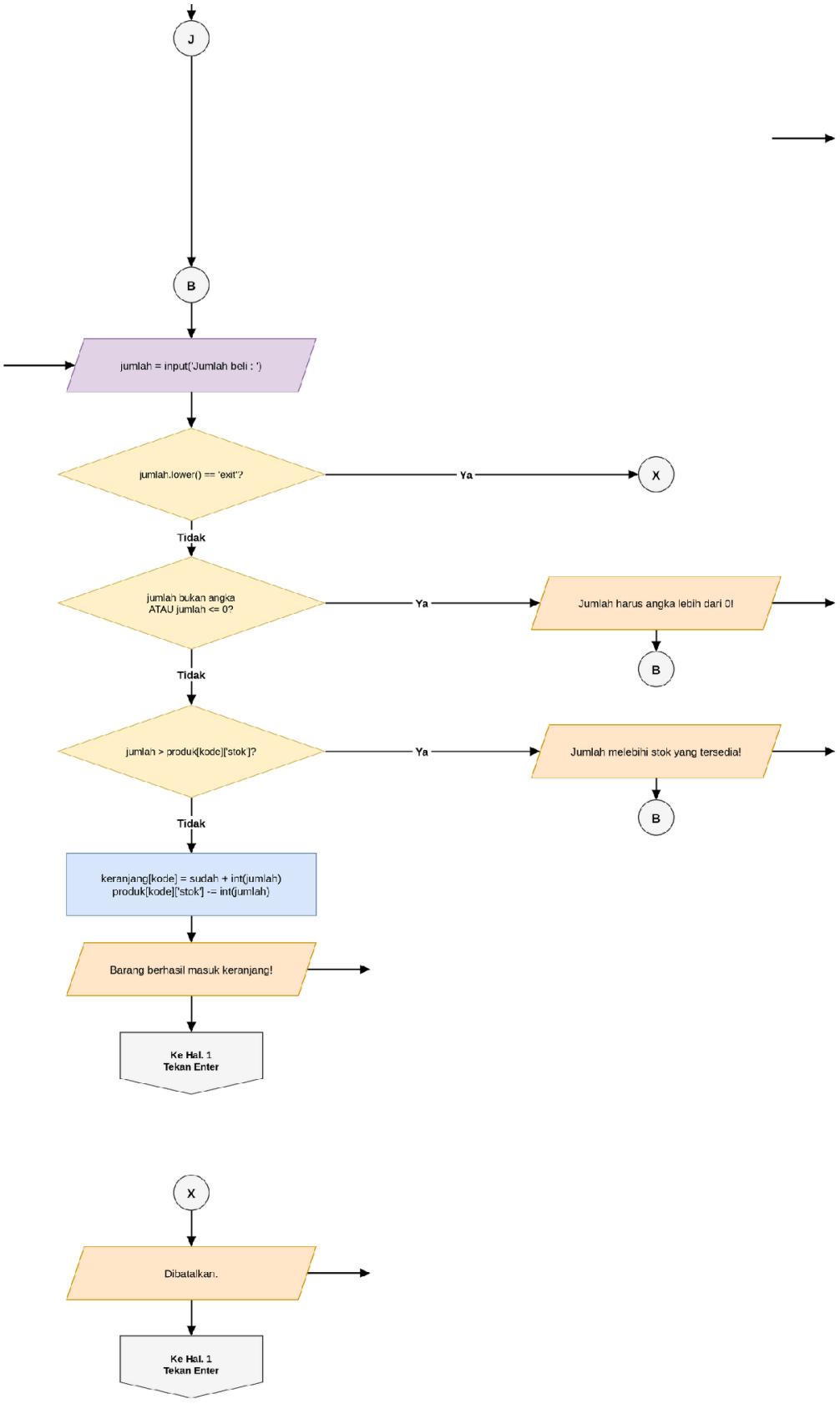
Gambar 1.10 *Flowchart Menu Pengguna (bagian 2 dari 2)*



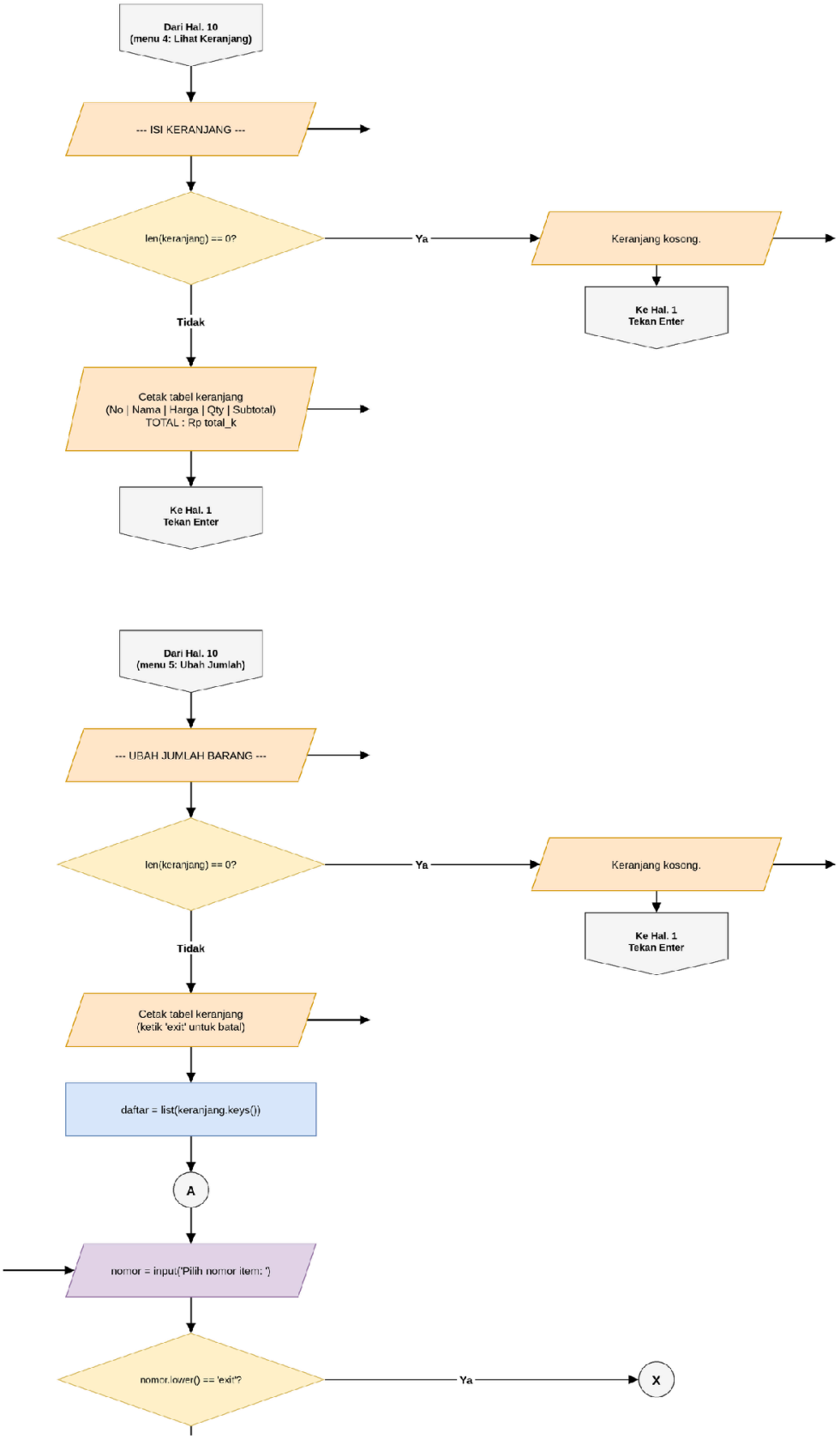
Gambar 1.11 *Flowchart User - Daftar & Cek Stok*



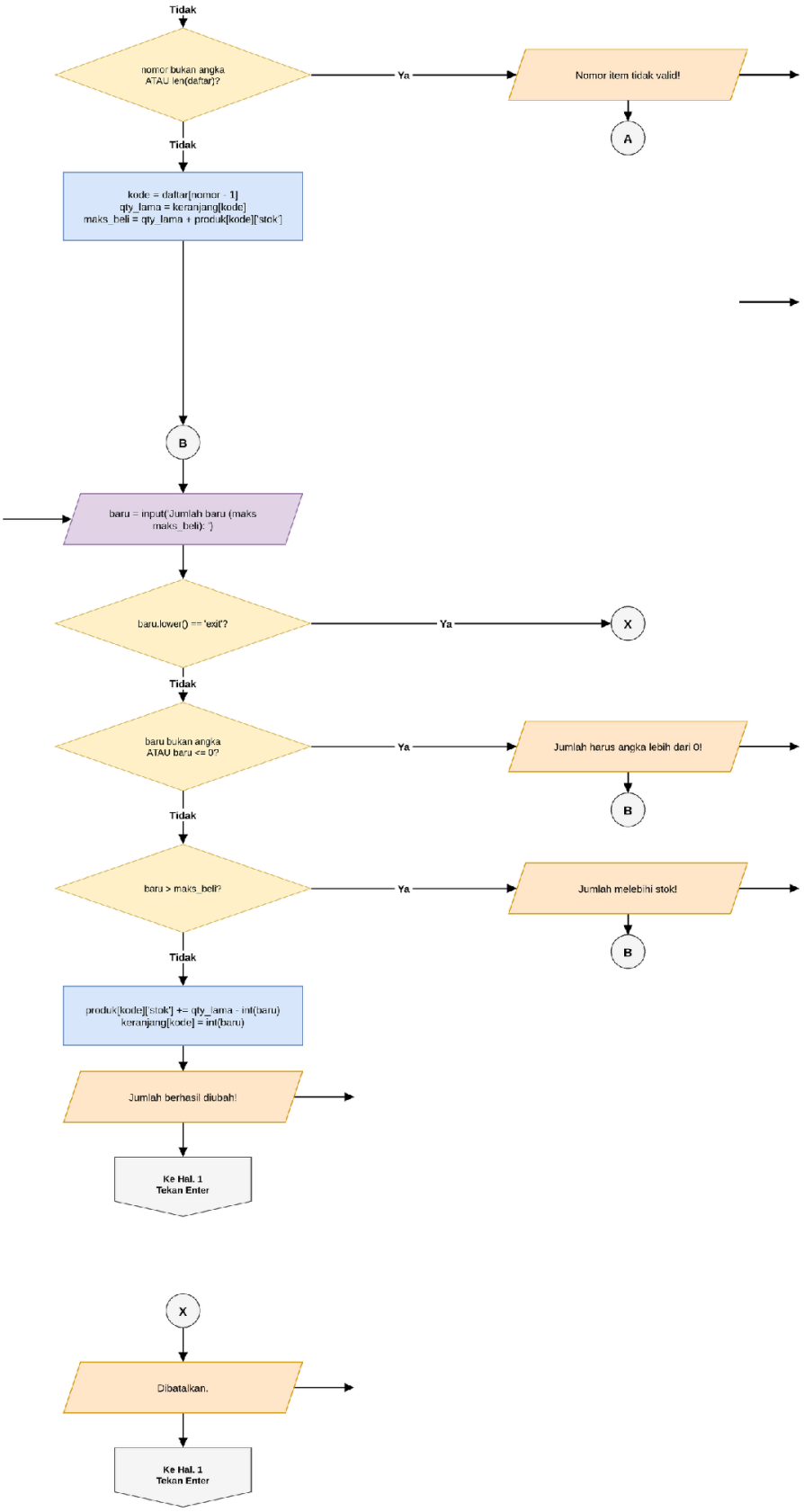
Gambar 1.12 *Flowchart User - Tambah Keranjang (bagian 1 dari 2)*



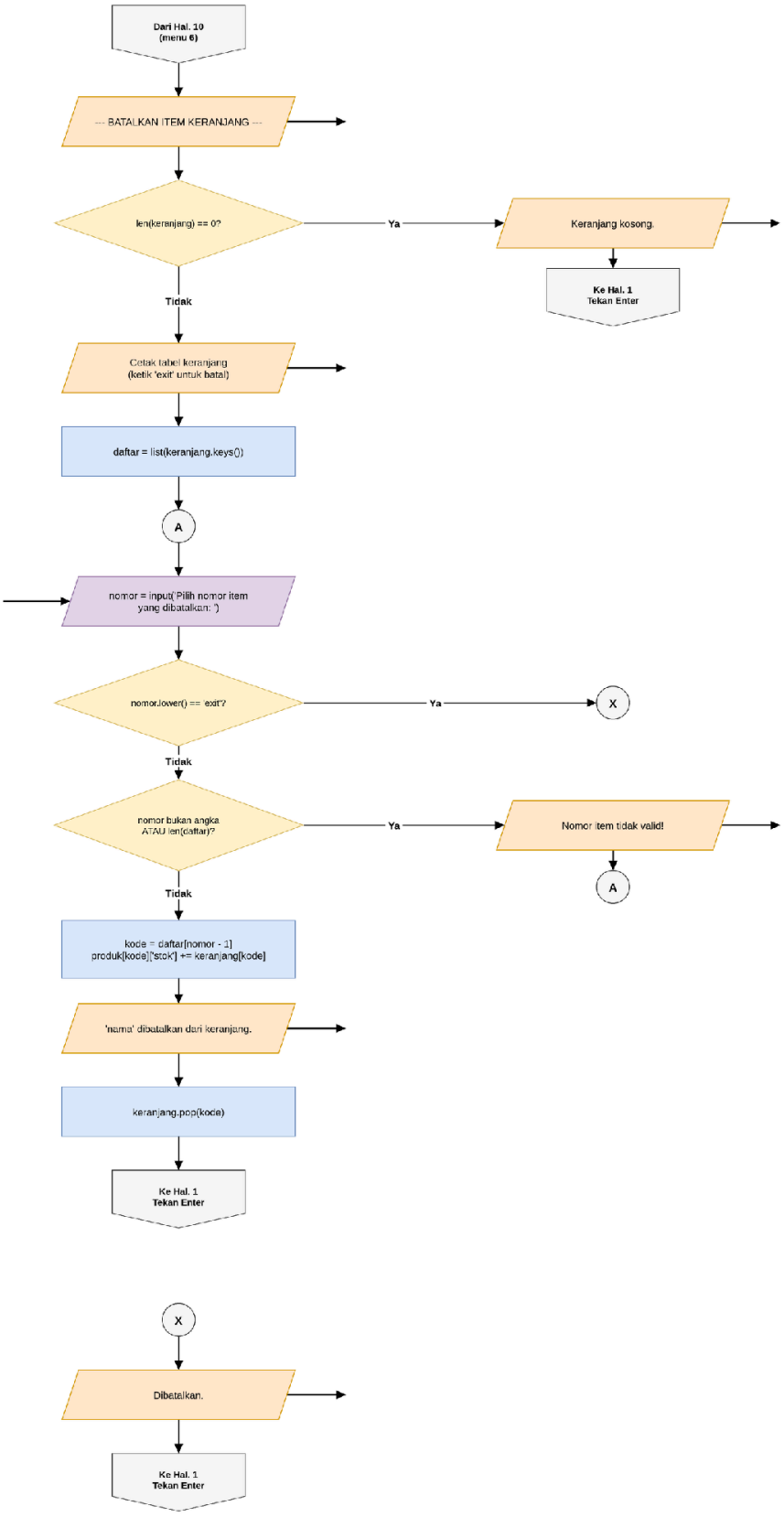
Gambar 1.12 *Flowchart User - Tambah Keranjang (bagian 2 dari 2)*



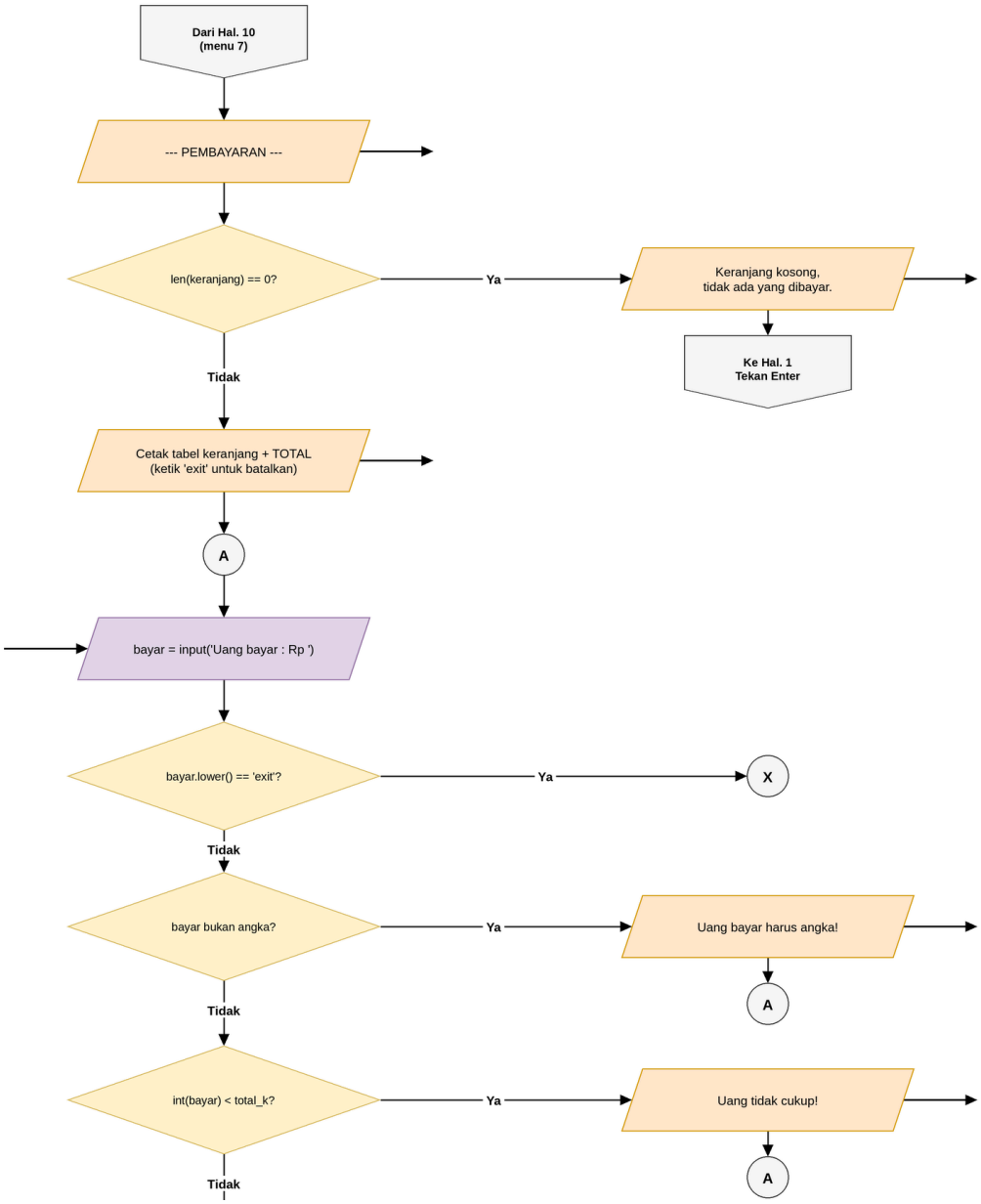
Gambar 1.13 *Flowchart User - Lihat & Ubah Keranjang (bagian 1 dari 2)*



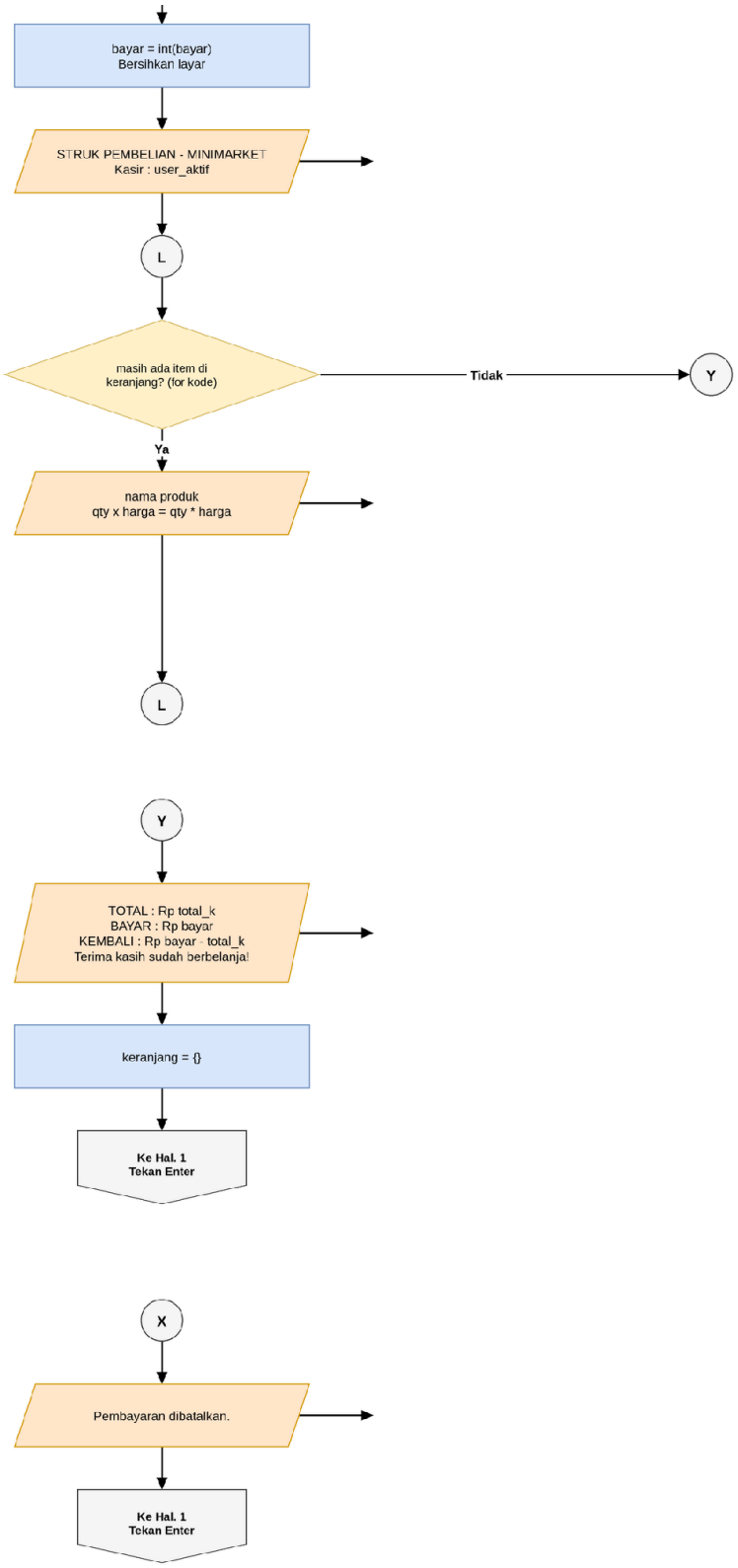
Gambar 1.13 *Flowchart User - Lihat & Ubah Keranjang (bagian 2 dari 2)*



Gambar 1.14 *Flowchart User - Batalkan Item*



Gambar 1.15 *Flowchart User - Bayar & Struk (bagian 1 dari 2)*



Gambar 1.15 *Flowchart User - Bayar & Struk (bagian 2 dari 2)*

2. Deskripsi Singkat Program

Program ini adalah sistem kasir minimarket berbasis terminal yang dibuat dengan Python. Pada Posttest 6 ini, penyimpanan data diubah menggunakan struktur data Dictionary (dict) untuk menyimpan akun users, daftar produk, dan isi keranjang, sehingga pengelolaan data lebih efisien. Semua data disimpan di memori dan akan hilang saat program ditutup.

Saat dijalankan, pengguna bisa memilih Login, Register, atau Keluar. Login dibatasi 3 kali percobaan. Register memvalidasi username (tidak kosong, tanpa spasi, belum dipakai) serta password minimal 4 karakter. Setelah login, menu yang muncul tergantung peran pengguna:

- **Admin** mengelola data produk: menambah produk dengan validasi kombinasi huruf dan angka, menampilkan daftar, mengecek stok, mengubah harga atau stok, dan menghapus produk.
- **Kasir/Pengguna** melayani pembelian: melihat produk, menambah barang ke keranjang berdasarkan stok, mengubah jumlah, membatalkan item, serta melakukan pembayaran dan mencetak struk.

3. Source Code

3. Source Code

A. Inisialisasi Data Dictionary

Pada versi ini, data pengguna, produk, dan keranjang disimpan dalam dictionary.

```
import os

users = {
        "admin": {"password": "admin17", "role": "admin"},
        "kasir": {"password": "kasir17", "role": "user"},
}

produk = {
        "P001": {"nama": "Indomie Goreng", "harga": 3500, "stok": 50},
        "P002": {"nama": "Aqua 600ml", "harga": 4000, "stok": 40},
        "P003": {"nama": "Roti Tawar", "harga": 15000, "stok": 10},
        "P004": {"nama": "Susu UHT 250ml", "harga": 6500, "stok": 25},
}

keranjang = {}
```

B. Fitur Login

Fitur ini memvalidasi username dan password yang ada di dalam dictionary users. Pengguna diberi 3 kali kesempatan untuk login.

```
if pilihan == "1":
        print("\n--- LOGIN ---")
        print("(ketik 'exit' untuk kembali)")
        percobaan = 0
        batal = False
        while percobaan < 3 and user_aktif == "":
        username = input("Username : ")
        if username.lower() == "exit":
        batal = True
        break
        password = input("Password : ")
        if password.lower() == "exit":
        batal = True
        break

        if username in users and users[username]["password"] == password:
        user_aktif = username
```

```
        keranjang = {}
        else:
        percobaan += 1
        print("Username atau password salah! Sisa:", 3 - percobaan)
```

C. Fitur Register

Digunakan untuk mendaftarkan akun baru dan memasukkannya ke dictionary users dengan role sebagai user.

```
elif pilihan == "2":
        print("\n--- REGISTER ---")
        print("(ketik 'exit' untuk kembali)")
        batal = False

        while True:
        username = input("Username baru : ")
        if username.lower() == "exit":
        batal = True
        break
        if username == "":
        print("Username tidak boleh kosong!")
        elif " " in username:
        print("Username tidak boleh ada spasi!")
        elif username in users:
        print("Username sudah dipakai!")
        else:
        break

        # ... (Validasi password)
        if not batal:
        users[username] = {"password": password, "role": "user"}
        print("Register berhasil! Silakan login.")
```

D. Fitur Tambah Produk (Admin)

Menyimpan produk baru ke dalam dictionary produk dengan validasi input yang ketat.

```
if not batal:
        produk[kode] = {"nama": nama, "harga": int(harga), "stok": int(stok)}
        print("\nProduk berhasil ditambahkan!")
else:
        print("\nTambah produk dibatalkan.")
```

E. Fitur Ubah & Hapus Produk (Admin)

Menggunakan pemanggilan key pada dictionary untuk mengubah nilai, dan fungsi .pop() untuk menghapus item.

```
# Ubah Harga
if not batal:
        produk[kode]["harga"] = int(baru)
        print("\nHarga berhasil diubah!")

# Hapus Produk
yakin = input("Yakin hapus '" + produk[kode]["nama"] + "'? (y/n): ")
if yakin.lower() == "y":
        produk.pop(kode)
        print("\nProduk berhasil dihapus!")
```

F. Fitur Tambah Barang ke Keranjang (Kasir)

Mengecek stok pada dictionary produk dan menyimpannya ke dictionary keranjang dengan key berupa kode barang.

```
if batal:
        print("\nDibatalkan.")
else:
        jml_beli = int(jumlah)
        keranjang[kode] = sudah + jml_beli
        produk[kode]["stok"] -= jml_beli
        print("\nBarang berhasil masuk keranjang!")
```

G. Fitur Pembayaran & Cetak Struk (Kasir)

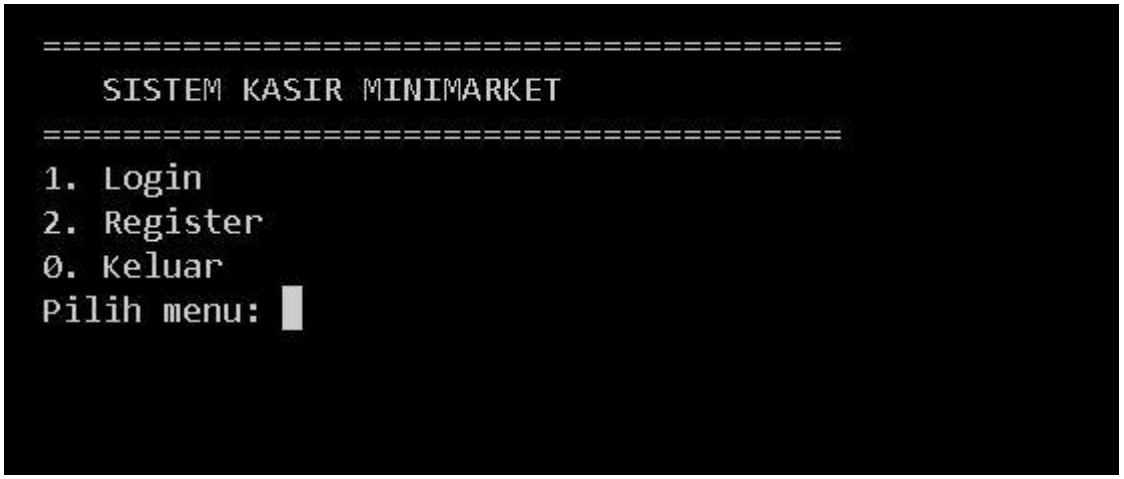
Menghitung subtotal dengan mengalikan harga pada dictionary produk dan kuantitas pada dictionary keranjang.

```
print("=" * 40)
print("          STRUK PEMBELIAN")
print("          MINIMARKET")
print("=" * 40)
print("Kasir :", user_aktif)
print("-" * 40)
for kode in keranjang:
        qty = keranjang[kode]
        harga = produk[kode]["harga"]
        print(produk[kode]["nama"])
        print("  ", qty, "x", harga, "=", qty * harga)
```

```
print("-" * 40)
print("TOTAL   : Rp", total_k)
print("BAYAR   : Rp", bayar)
print("KEMBALI : Rp", bayar - total_k)
print("=" * 40)
print("   Terima kasih sudah berbelanja!")
keranjang = {}
```

4. Hasil Output

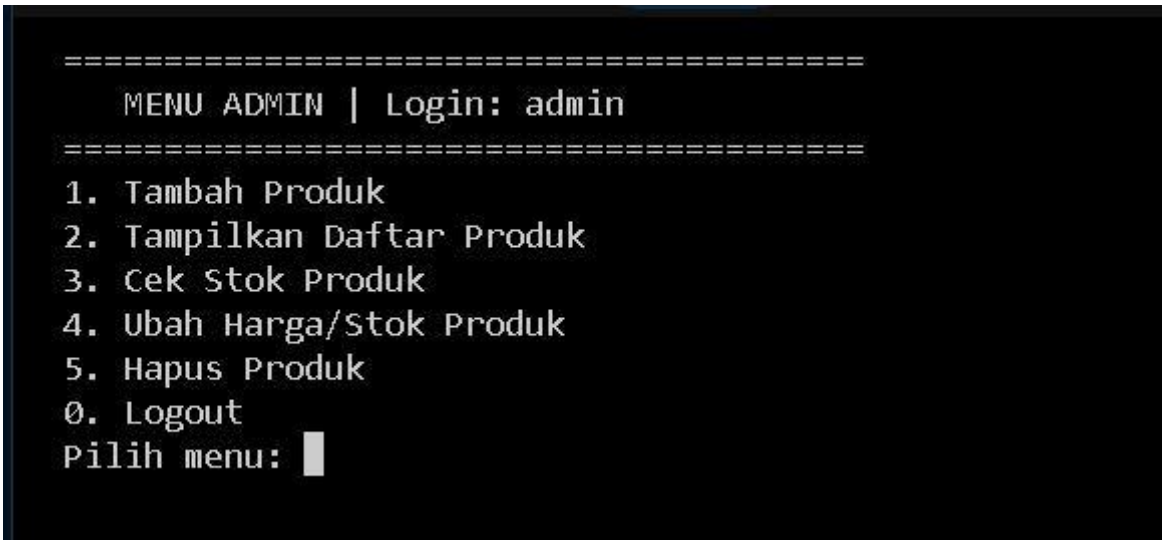
4. Hasil Output



Gambar 4.1 Tampilan Login, Register dan Keluar



Gambar 4.2 Login Admin



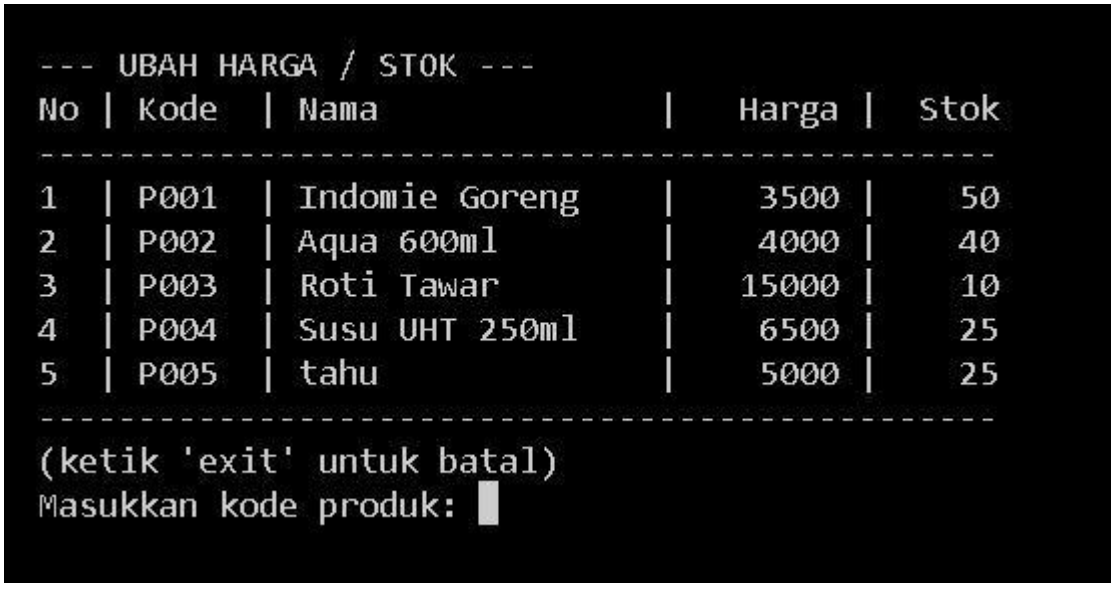
Gambar 4.3 Tampilan Menu Admin



Gambar 4.4 Tampilan Daftar Produk



Gambar 4.5 Tampilan Cek Stok



Gambar 4.6 Tampilan Ubah Harga/Stok

```
--- HAPUS PRODUK ---
No | Kode  | Nama              |   Harga |  Stok
-------------------------------------------------
1  | P001  | Indomie Goreng    |    3500 |    50
2  | P002  | Aqua 600ml        |    4000 |    40
3  | P003  | Roti Tawar        |   15000 |    10
4  | P004  | Susu UHT 250ml    |    6500 |    25
5  | P005  | tahu              |    5000 |    25
-------------------------------------------------
(ketik 'exit' untuk batal)
Masukkan kode produk yang dihapus:
```

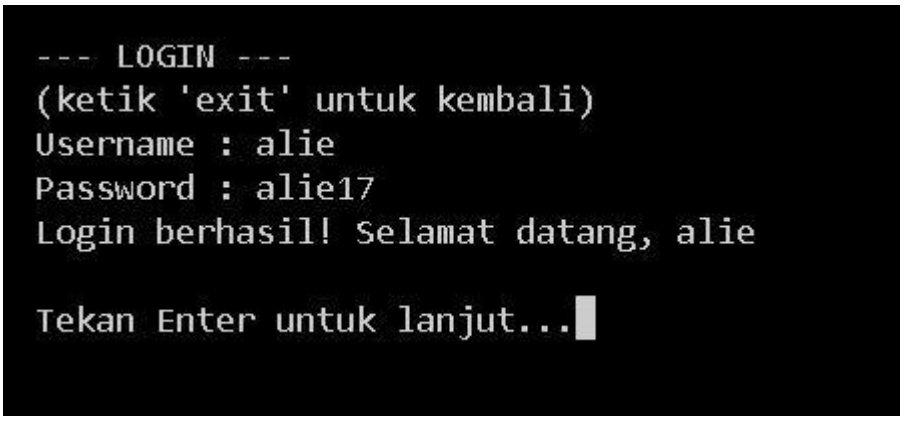
Gambar 4.7 Tampilan Hapus Produk

```
1. Login
2. Register
0. Keluar
Pilih menu: 2

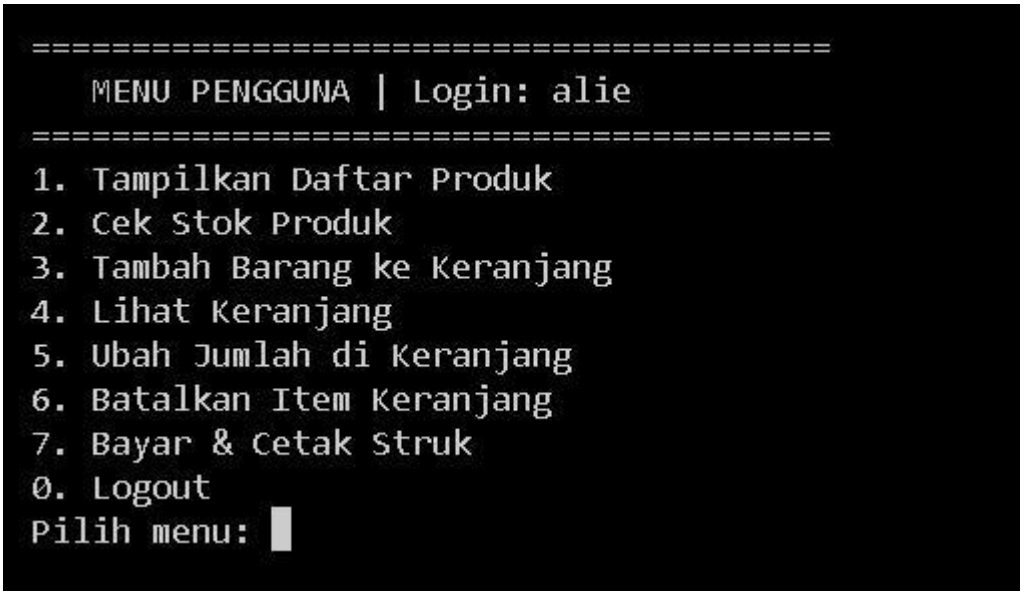
--- REGISTER ---
(ketik 'exit' untuk kembali)
Username baru : al kahfi
Username tidak boleh ada spasi!
Username baru : alie
Password baru : alie17
Register berhasil! Silakan login.

Tekan Enter untuk lanjut...
```

Gambar 4.8 Tampilan Menu Register



Gambar 4.9 Tampilan Tampilan Login User



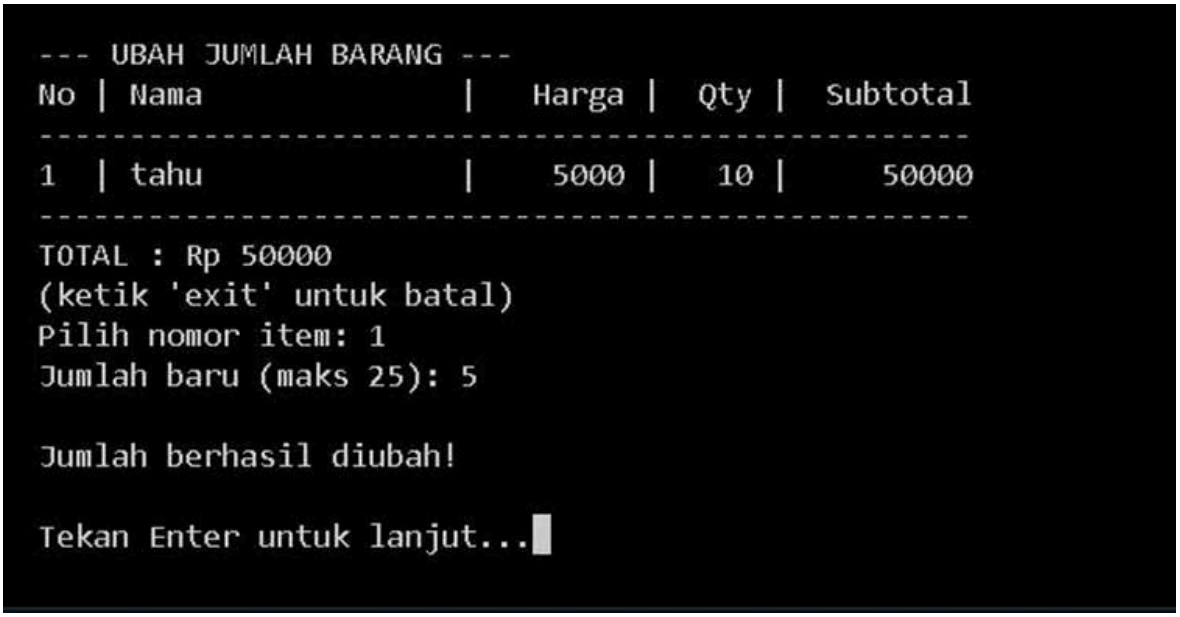
Gambar 4.10 Tampilan Tampilan Menu User



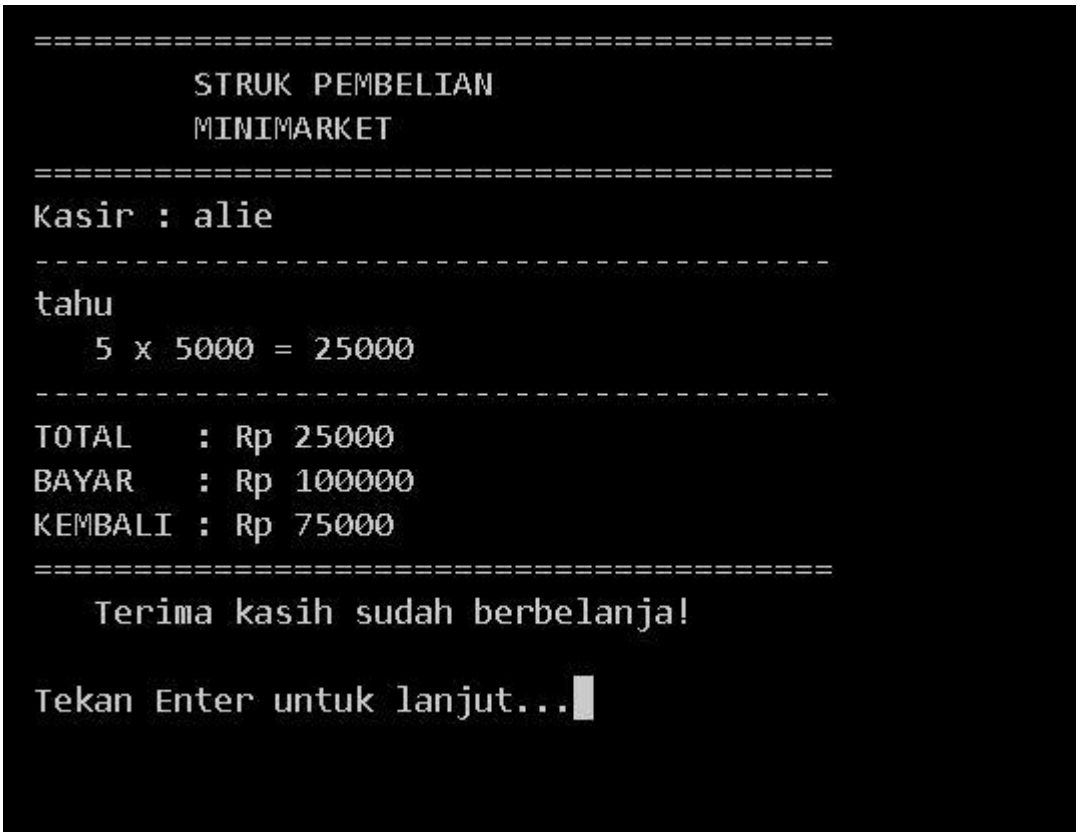
Gambar 4.11 Tampilan Tampilan User Tambah Ke Keranjang



Gambar 4.13 Tampilan Tampilan Hasil Lihat Keranjang



Gambar 4.14 Tampilan Tampilan Ubah Keranjang



Gambar 4.15 Tampilan Tampilan Hasil Cetak Struk

5. Langkah-langkah Git

5. 1 Git Add

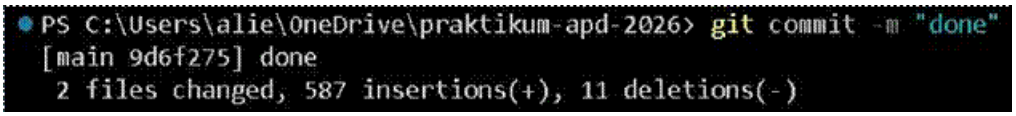
Perintah git add digunakan untuk menambahkan file baru atau perubahan file ke dalam staging area Git sebelum disimpan melalui commit. Perintah ini dapat digunakan untuk memilih file tertentu atau menambahkan semua perubahan.



***Gambar 5.1** Git Add*

5.2 Git Commit

Perintah git commit digunakan untuk menyimpan perubahan yang sudah dimasukkan ke staging area sebagai sebuah riwayat perubahan di repositori lokal. Setiap commit dapat diberi pesan untuk menjelaskan perubahan yang dilakukan.



Gambar 5.2 *Git Commit*

5.3 Git Push

Perintah git push digunakan untuk mengirimkan commit dari repositori lokal ke repositori jarak jauh, misalnya GitHub. Dengan perintah ini, perubahan program dapat diunggah dan disimpan di repositori online.



Gambar 5.3 *Git Push*